

Drone hybride à ailes fixes : un meilleur rendement qu'un drone classique ?



Introduction / Motivation



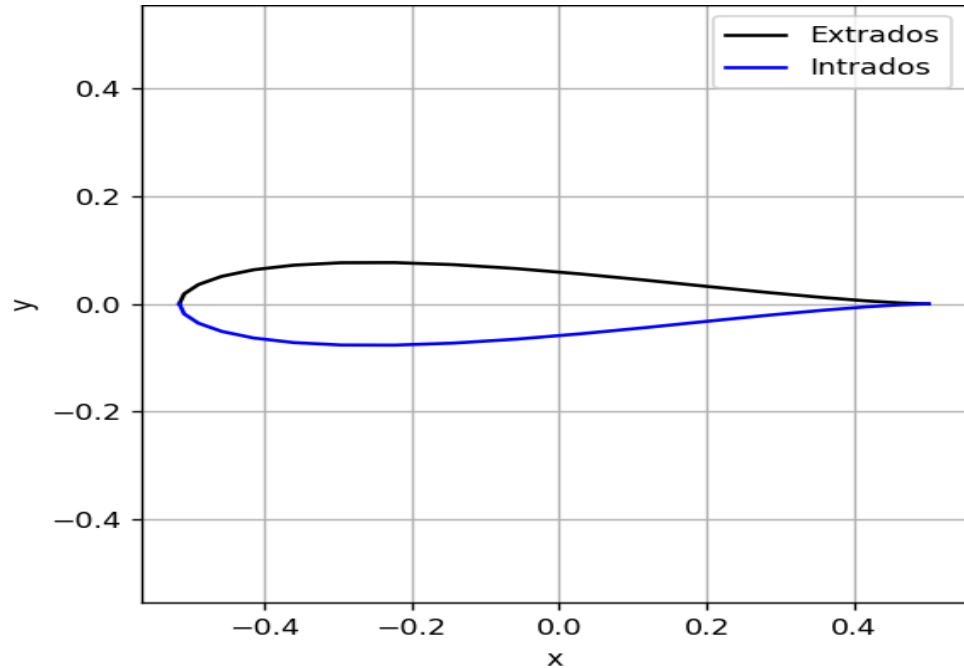
- Autonomie moyenne faible (15–30 min)
- Énergie utilisée uniquement pour compenser le poids
- Portance générée uniquement par les rotors
- Crash très punitif

Vue d'ensemble

- I) Principes des ailes d'avions & Etude
 - Profil et angles d'attaques
 - Angle optimal /Simulation
- II)Application théorique à un drone hybride
 - Statique , Dynamique
 - Simulations

I) Principes: Comment une aile permet le vol

- Profil de l'aile étudié:

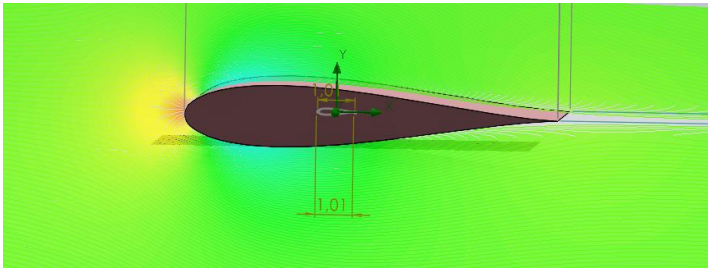
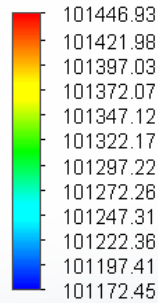
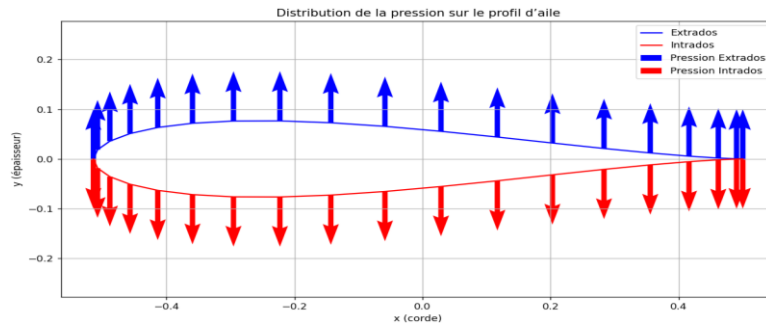


- Forces :
 - Portance : $F_p = \frac{1}{2} \cdot \rho \cdot \text{vitesse de l'air}^2 \cdot \text{Surface alaire} \cdot \text{Coefficient de portance}$
 - Trainée : $F_p = \frac{1}{2} \cdot \rho \cdot \text{vitesse de l'air}^2 \cdot \text{Surface alaire} \cdot \text{Coefficient de trainée}$

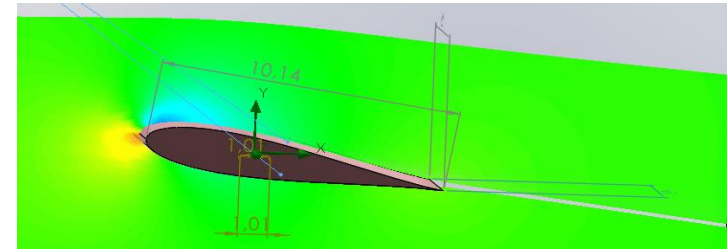
avec ρ la masse volumique de l'air : $1,225 \text{ kg/m}^3$

Importance de l'angle d'attaque

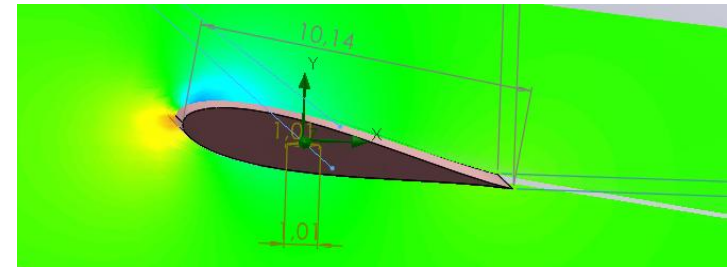
Aile symétrique à 0 degré



Aile symétrique à 9 degrés

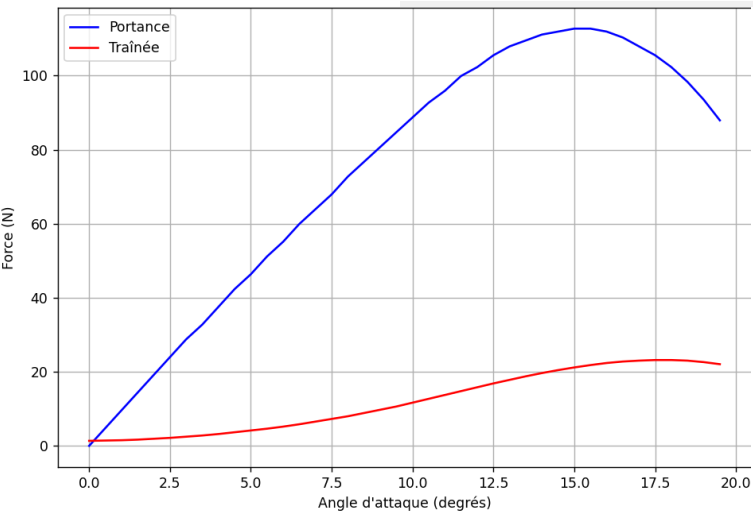
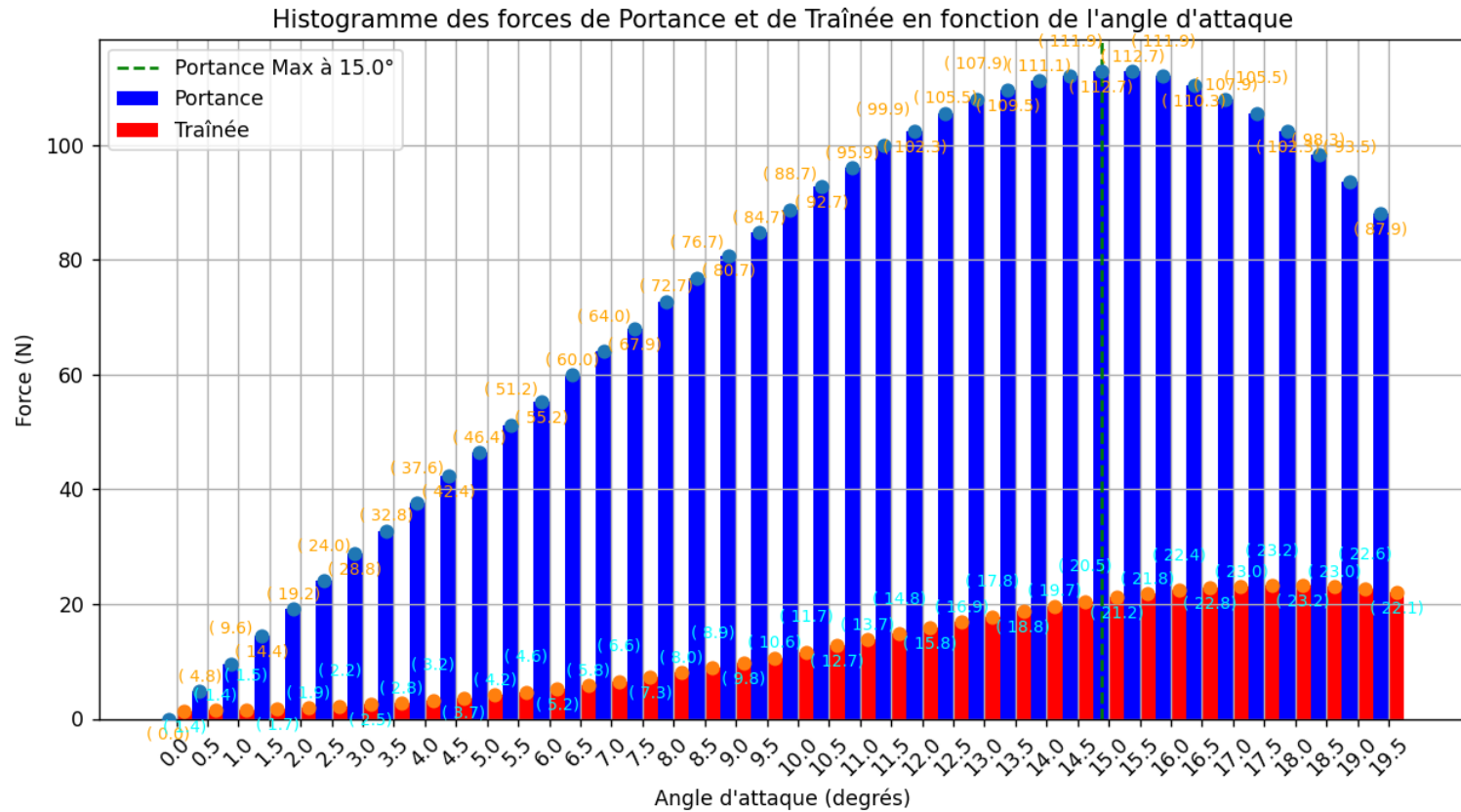


Aile symétrique à 12 degrés

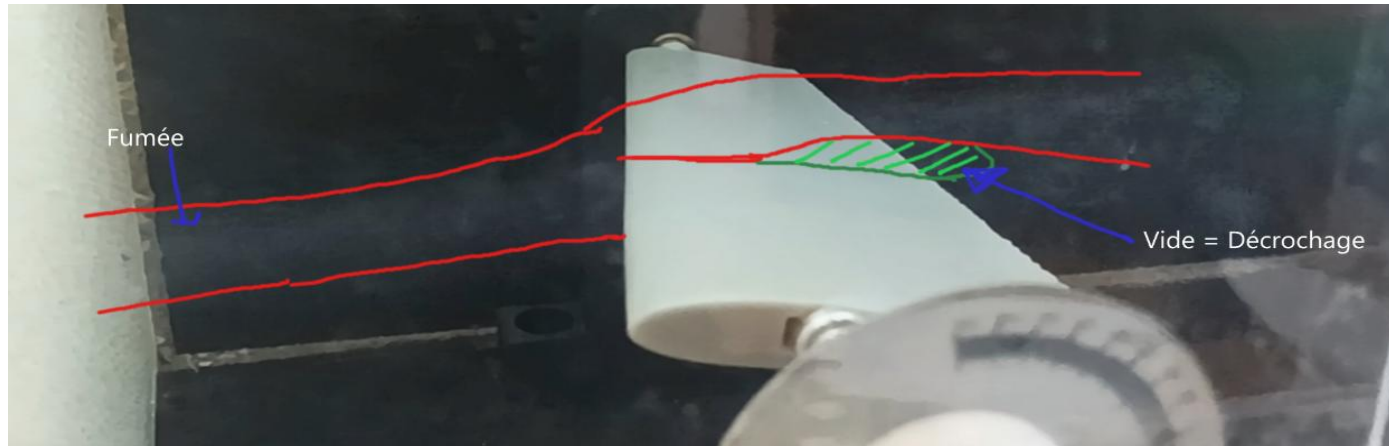


Portance (N)	Trainée (N)	Angle d'attaque (°)
0	0	0
1,6	0,3	9
1,68	0,4	12
1,8	0,6	15

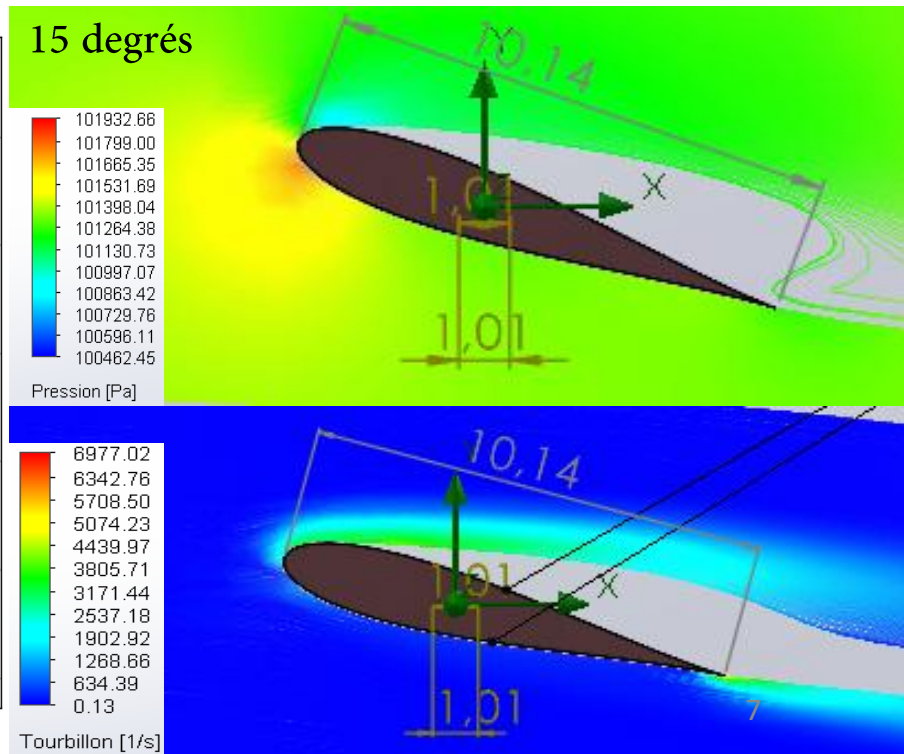
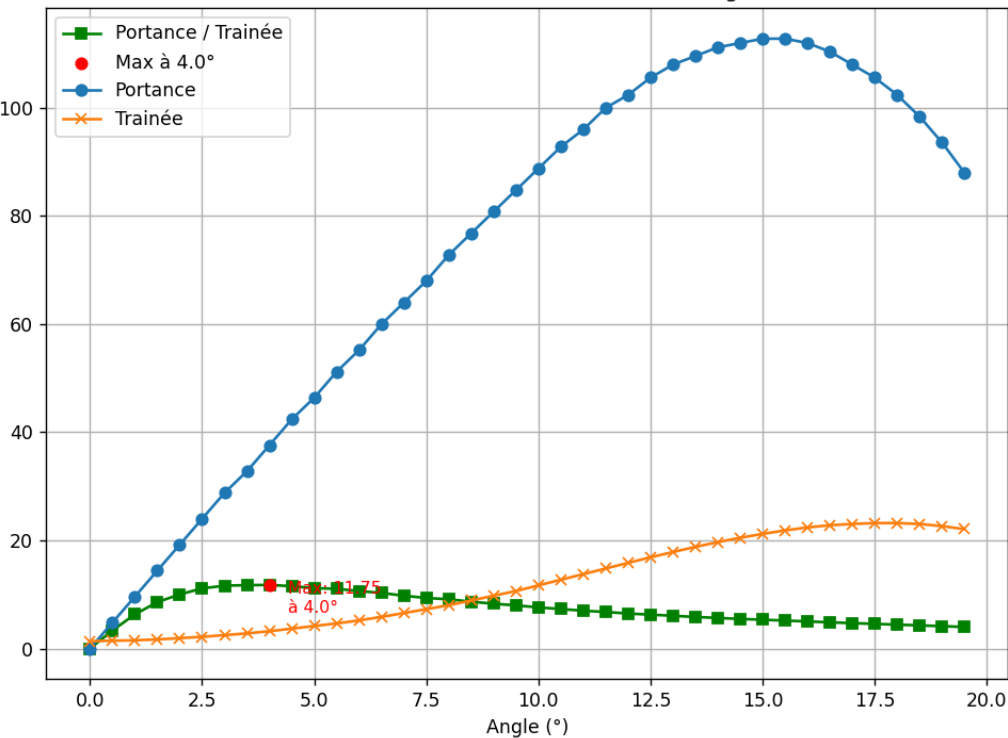
Modélisation & limites



Décrochage

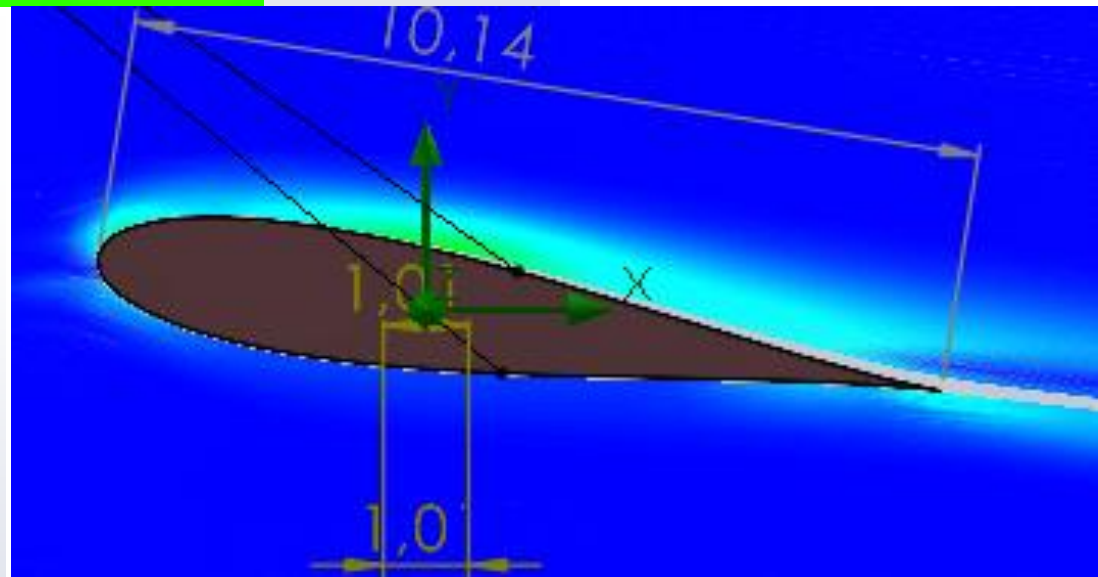
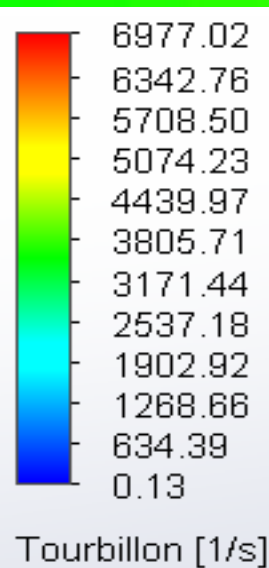
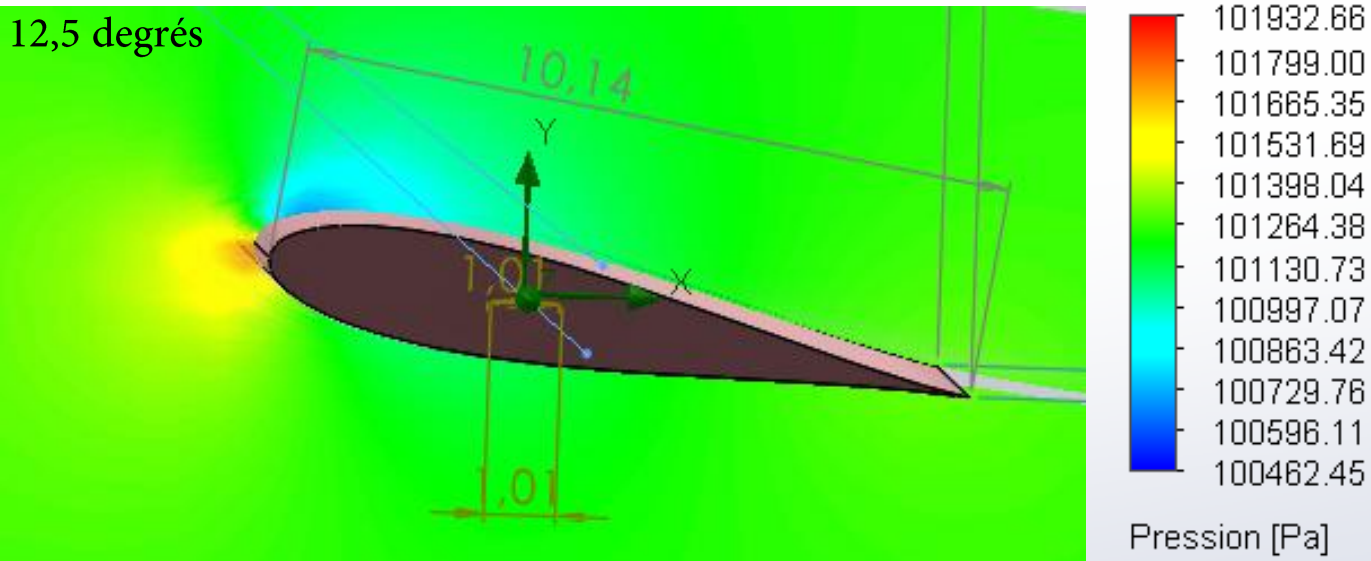


Portance et Trainée en fonction de l'angle

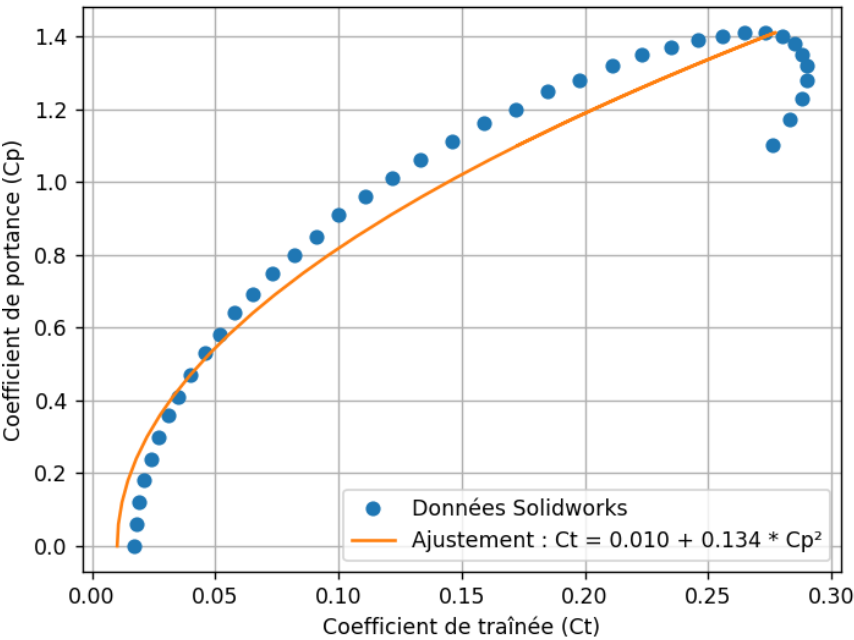


Angle optimal

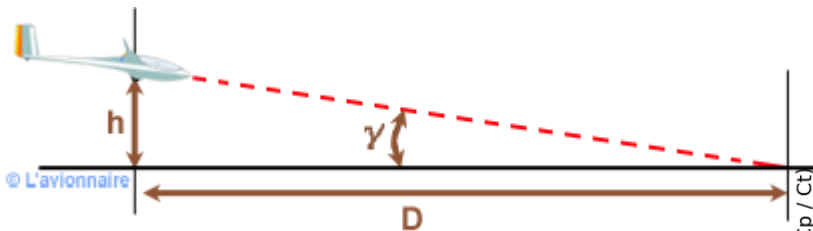
12,5 degrés



Généralisation des coefficients

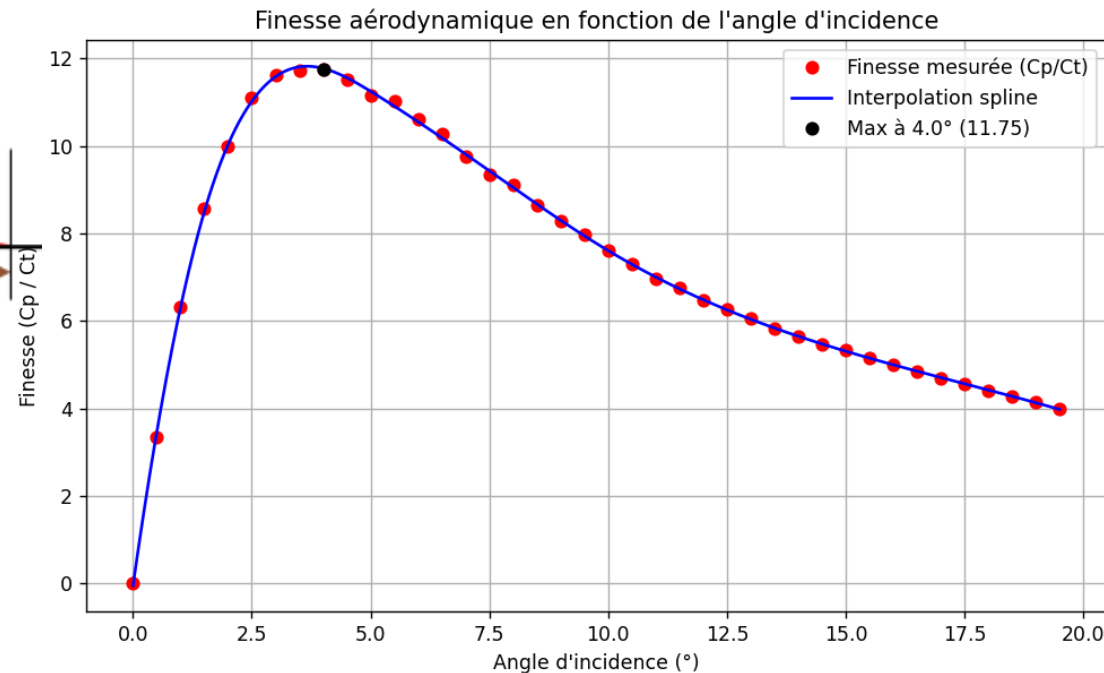


$$C_t = 0,01 + 0,134 * C_p^2$$



$$\text{finesse} = D/h = C_p/C_t = 6.2$$

à 12.5°



Intérêt de la finesse sur la chute

Chute assisté Drone hybride :

$$\text{Finesse} = 6.1 = D/h,$$

$$h=120, D = 6.1 \times 120 = 732 \text{ mètres}$$

$$V_x = D/t \text{ et } V_y = h/t \text{ on a } V = (V_x^2 + V_y^2)^{1/2}$$

$$\text{D'où } T_{\text{chute}} = (D^2 + h^2)^{1/2} / V$$

$$V = 10 \text{ m/s}$$

$$T_{\text{chute}} = 74.18 \text{ s}$$

Chute libre (Drone multi rotors) :

$$V(t) = -g \cdot t + V_0$$

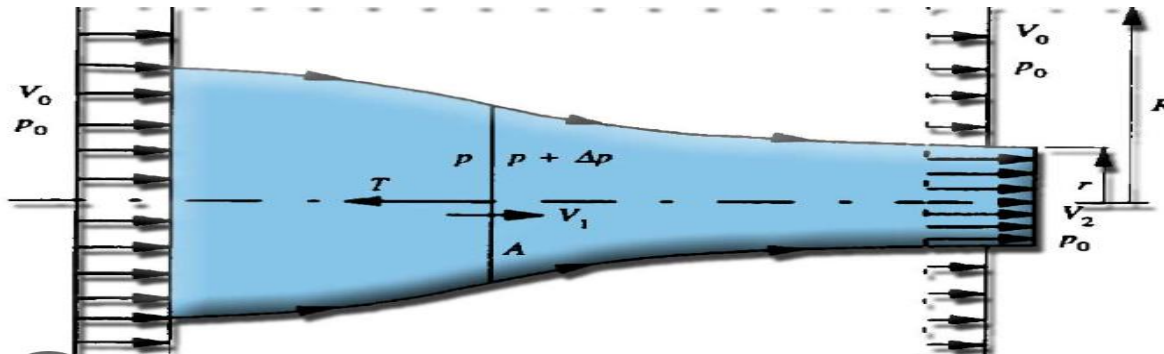
$$V_0 = 0 \text{ m/s}$$

$$Z(t) = -1/2 \cdot g \cdot t^2$$

$$T_{\text{chute}} = (2h/g)^{1/2} = 4.95 \text{ s}$$

Rotors : Force de propulsion

Théorie des disques (de Froude):

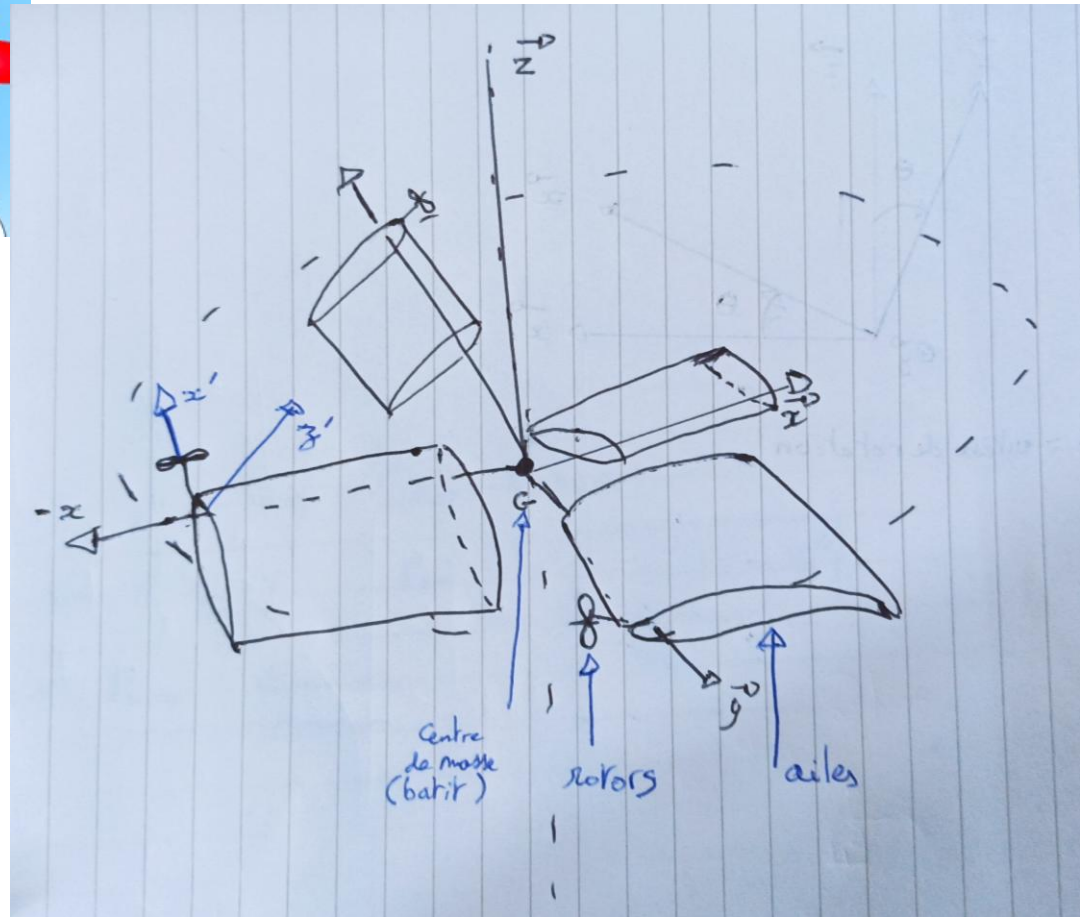


$F_{\text{propulsion}} = \rho \cdot \text{Surface du disque créée par les hélices des rotors} \cdot \text{Vitesse moyenne de l'air entrant} \cdot (\text{Vitesse de l'air sortant} - \text{Vitesse de l'air entrant})$

Puissance des rotors est de : $P_{\text{hélice}} = F_p \cdot \text{Vitesse moyenne de l'air entrant}$

Le pari hybride : et si on utilisait des ailes ?

- Question : *Les ailes peuvent-elles vraiment améliorer l'autonomie ?*



Lois physiques

- Principe fondamental de la dynamique
- Théorème de l'Énergie Cinétique via simulation numérique

À l'équilibre
 $\sum \vec{F} = 0$ donc $4(\vec{F}_{\text{propulsion}} + \vec{F}_{\text{trainé}} + \vec{F}_{\text{portance}}) + \vec{P}_{\text{oid}} = 0$

On a

$$\vec{F}_{\text{propulsion}} = F_{\text{prop}} \cdot \vec{e}_{x'}$$

$$\vec{F}_{\text{trainé}} = -F_{\text{trainé}} \cdot \vec{e}_{x'}$$

$$\vec{F}_{\text{portance}} = F_{\text{portance}} \cdot \vec{e}_{z'}$$

$$\vec{P}_{\text{oids}} = -P_{\text{oid}} \cdot \vec{e}_z$$

$$x' = \cos(\theta)x + \sin(\theta)z$$

$$z' = -\sin(\theta)x + \cos(\theta)z$$

$$(F_{\text{propulsion}} - F_{\text{trainé}}) \cos(\theta) = 4 F_{\text{portance}} \sin(\theta)$$

selon x

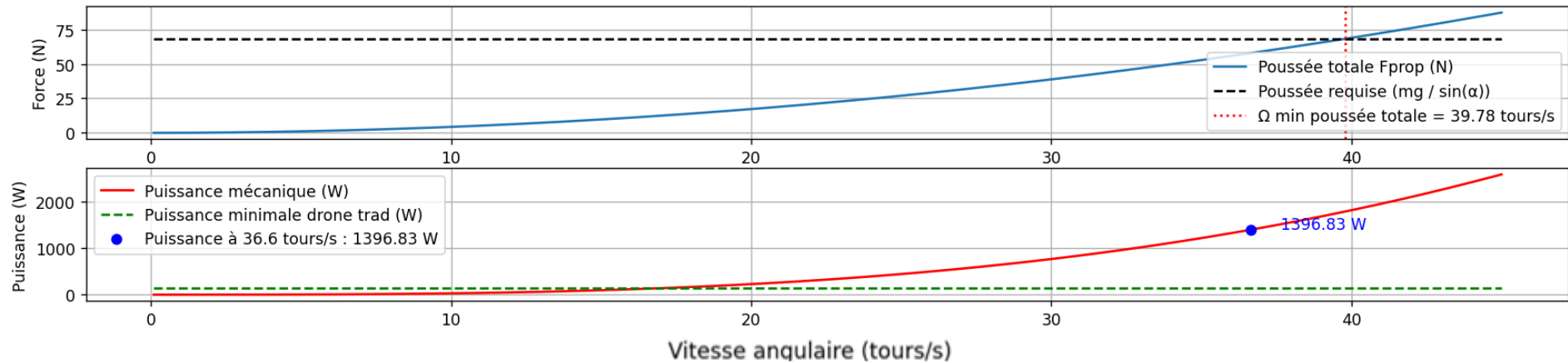
$$4 F_{\text{portance}} \cos(\theta) = M_{\text{drone}} g$$

selon z

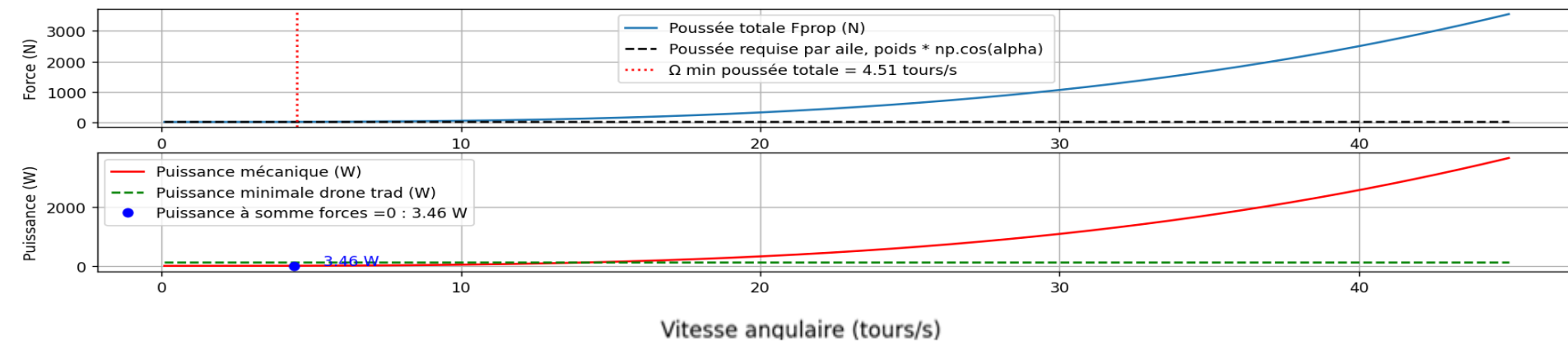
Etudes d'ailes via simulation python + Tec :

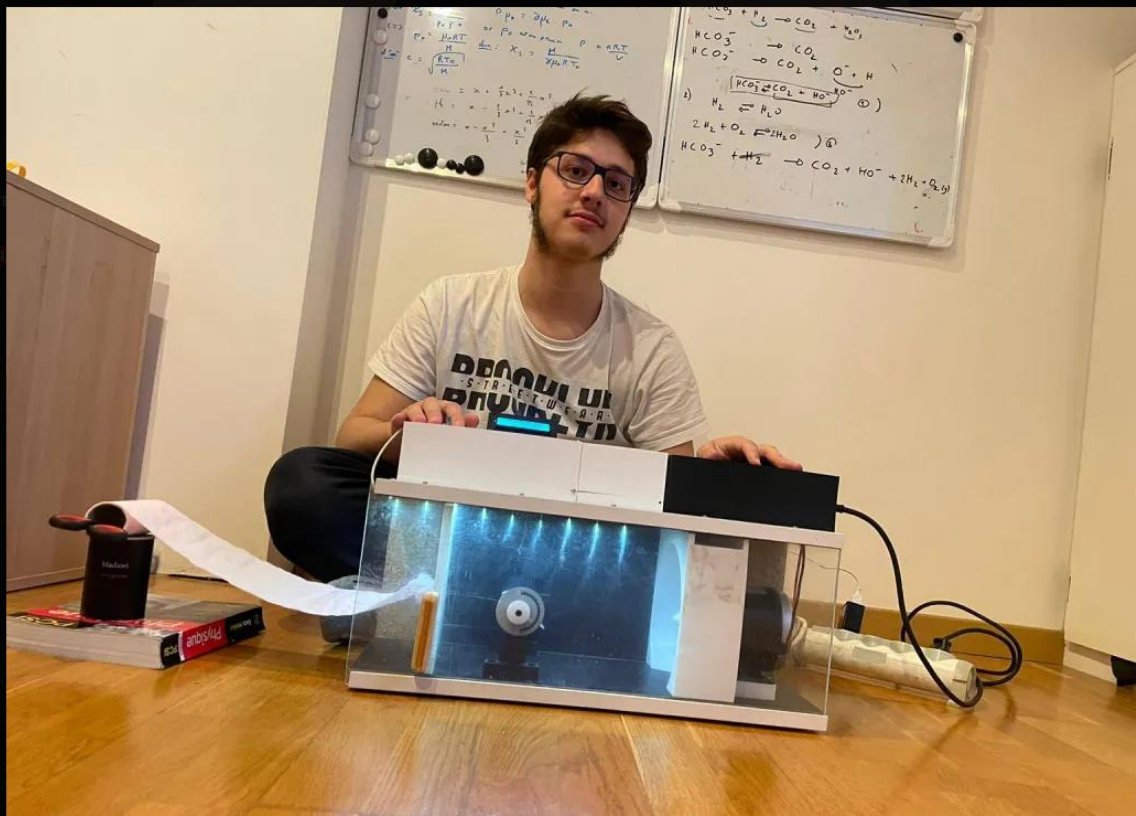
Drone masse=1.5 kg, angle $\alpha=12.5^\circ$ - Analyse puissance & pertes absolues

Drone traditionnel avec rotors inclinés selon alpha



Drone hybride avec rotors et ailes inclinés selon alpha





Merci de votre attention

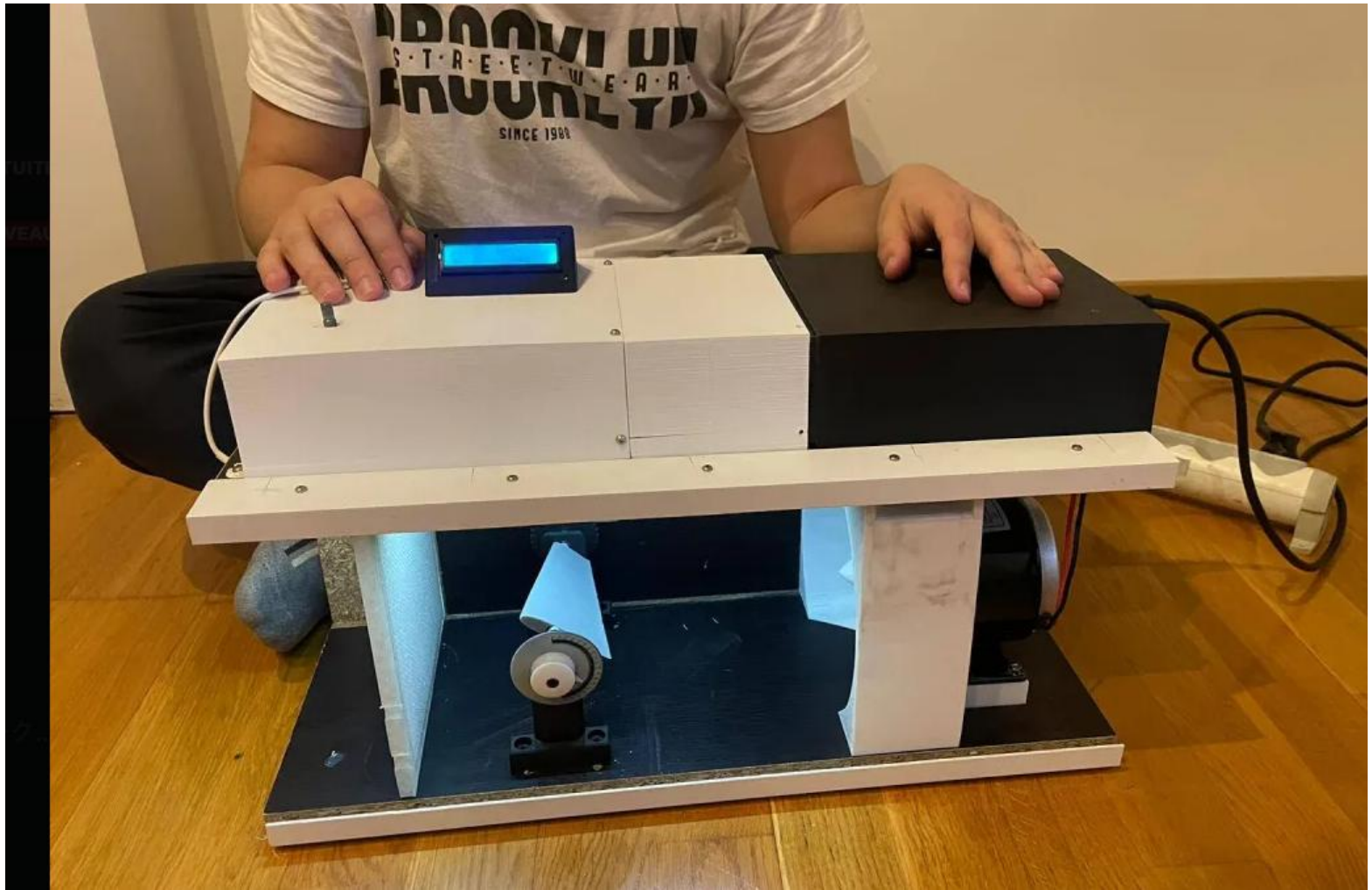


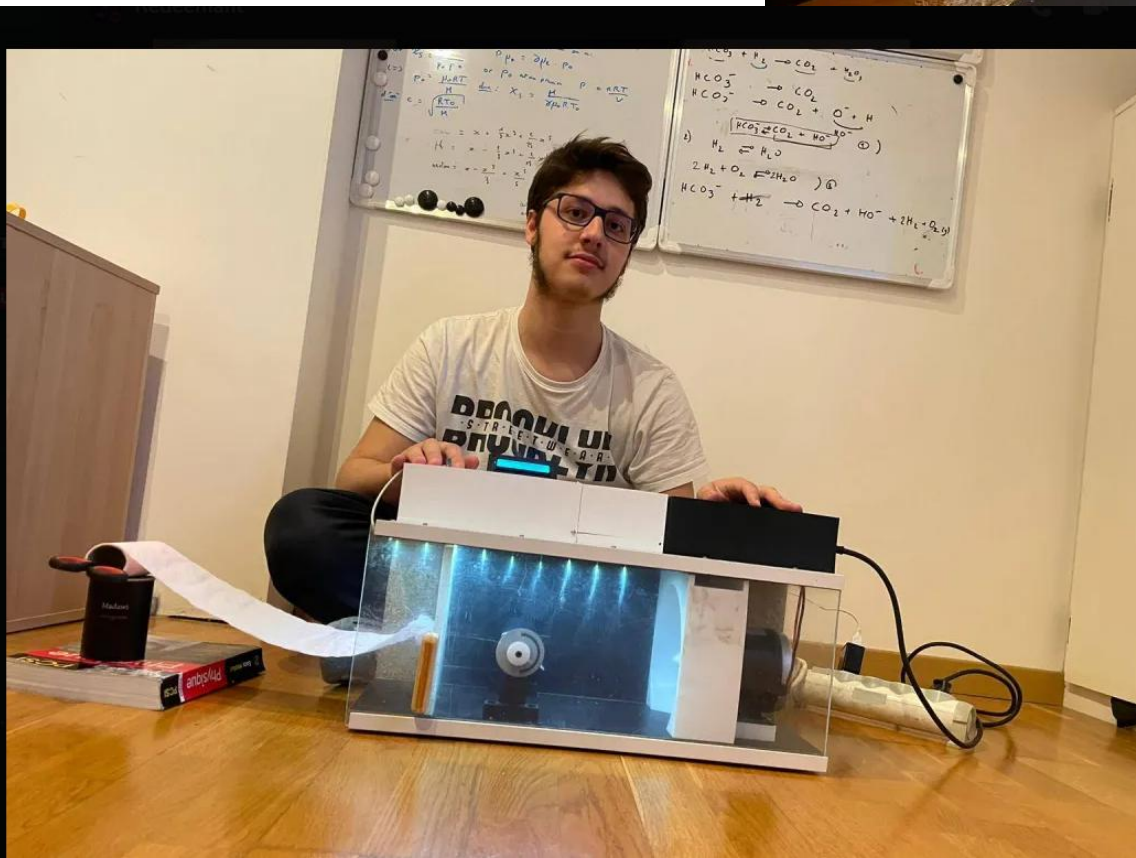
Supplément Photos :

- Soufflerie réelle :









Annexes pythons :

Drone avec aile symétrique 12 degrés

```
import numpy as np
import matplotlib.pyplot as plt

# Constantes
rho = 1.225      # Masse volumique air (kg/m3)
r = 0.15         # Rayon hélice (m)
R=0.5           # rayon du disque aile
A = np.pi * r**2 # Surface disque propulsif (m²)
m = 1.5         # Masse drone (kg)
g = 9.81        # Gravité (m/s²)
alpha_deg = 12.5 # Angle d'inclinaison de la poussée (°)
alpha = np.radians(alpha_deg)
L=0.2044 #perimetre de l aile m

S= R*L          #mcarré a augmenter plus tard
print("surface de l aile :",S,"mètres")
print("surface total :",4*S,"mètres carrés soit ")
# apres un PFS on trouve que [ (Fprop -Ftrain)=Fport*tan(alpha)      ET 4*Fport*(1/cos(15))=Poids
#coeff de trainée et de portance à 15 deg : Cp=1.41 & Ct=0.265
```

```
# Gamme vitesse angulaire (tours/s)
omega_rps = np.linspace(0.1, 45, 500) # tours/s
omega = omega_rps * 2 * np.pi         # rad/s
```

```
# Hypothèses simplifiées : V0 et Ve proportionnels à omega * r
k1 = 0.3
k2 = 0.8
```

```
Cp = 1.41      # Coefficient de portance à 12.5°
Ct = 0.265     # Coefficient de trainée à 12.5°
```

```
def vitesse_V0(omega):
    return k1 * omega * r
```

```
def vitesse_Ve(omega):
    return vitesse_V0(omega) + k2 * omega * r
```

```
# Vitesse moyenne induite par les hélices
```

```
def vitesse_moyenne(omega):
    return (vitesse_V0(omega) + vitesse_Ve(omega)) / 2
```

```
def F_portance(omega):
    V = vitesse_moyenne(omega)
    return 0.5 * rho * S * V**2 * Cp
#print(F_portance(omega), "portance N")
```

```
def pousse(omega):
    V0 = vitesse_V0(omega)
    Ve = vitesse_Ve(omega)
    vi = (V0 + Ve) / 2
    return rho * A * vi * (Ve - V0), vi, Ve # retourne poussée et vitesse moyenne
```

```
def puissance_mecanique(omega):
    Fprop = F_propu_requise(omega)
    _,vi, Ve=pousse(omega)
    Vi = vitesse_moyenne(omega)
    Fport = F_portance(omega)
    Ftrain = F_trainee(omega)
    return (Fport * np.cos(alpha) * Ve + (Fprop - Ftrain) * np.sin(alpha) * vi)
```

```
def puissance_minimale(omega):
    Pmin = (m * g) ** 1.5 / np.sqrt(2 * rho * A)
    return Pmin
print("puissance minimal est de ",puissance_minimale(alpha),"W")
```

```
def perte_energetique(omega):
    P_meca = puissance_mecanique(omega)
    P_min = puissance_minimale(omega)
```

```

P_perte = P_meca - P_min
return P_perte

def calcul_puissances(omega):
    P_meca_list = []
    P_min_list = []
    rendement=[]

    for w in omega:
        P_meca = puissance_mecanique(w)
        P_min = puissance_minimale(w)
        P_min_list.append(P_min)
        P_meca_list.append(P_meca)
        rendement.append((1-(P_min/P_meca))*100)

    return rendement

#print (omega )    # okay

rendement_percent = calcul_puissances(omega)
#for i in range(len(omega)):
    # print(f"omega[{i}] = {omega[i]:.2f}, rendement = {rendement_percent[i]:.2f}%")

```

```

# Initialisation des tableaux
Fprop_arr = np.zeros_like(omega)
vi_arr = np.zeros_like(omega)
P_meca_arr = np.zeros_like(omega)
P_min_arr = np.zeros_like(omega)
P_perte_arr = np.zeros_like(omega)
Fport_arr = np.zeros_like(omega)
Ftrain_arr = np.zeros_like(omega)

# Calculs pour chaque vitesse angulaire
for i, w in enumerate(omega):
    Fprop_arr[i], vi_arr[i], _ = pousse(w)
    P_meca_arr[i] = puissance_mecanique(w)
    P_min_arr[i] = puissance_minimale(w)
    P_perte_arr[i] = -perte_energetique(w)
    Fport_arr[i] = F_portance(w)
    Ftrain_arr[i] = F_trainee(w)

```

```

# Poussée requise corrigée par l'angle
poids = m * g
F_req = poids * np.cos(alpha)/4

# Composante verticale de la poussée
_,vi,Ve=pousse(omega)
Tz = Fport_arr * np.cos(alpha) * Ve + (Fprop_arr - Ftrain_arr) *np.sin(alpha)

# Recherche des vitesses angulaires minimales
idx_Fprop = np.where(Tz >= F_req)[0]
omega_Fprop_min = omega_rps[idx_Fprop[0]] if len(idx_Fprop) > 0 else None

idx_Tz = np.where(Tz >= F_req)[0]
omega_Tz_min = omega_rps[idx_Tz[0]] if len(idx_Tz) > 0 else None

```

```

# Affichage des courbes
plt.figure(figsize=(14, 12))

plt.subplot(5,1,1)
plt.plot(omega_rps, vitesse_V0(omega), label='Vitesse amont V0 (m/s)')
plt.plot(omega_rps, vitesse_Ve(omega), label='Vitesse sortie Ve (m/s)')
plt.ylabel('Vitesse (m/s)')
plt.legend()
plt.grid()

plt.subplot(5,1,2)
plt.plot(omega_rps, Tz, label='Poussée totale Fprop (N)')
plt.plot(omega_rps, np.full_like(omega_rps, F_req), 'k--', label='Poussée requise par aile, poids * np.cos(alpha) ')
if omega_Fprop_min is not None:
    plt.axvline(omega_Fprop_min, color='r', linestyle=':', label=f'Q min poussée totale = {omega_Fprop_min:.2f} tours/s')
plt.ylabel('Force (N)')
plt.legend()
plt.grid()

# Trouver l'indice où Tz est le plus proche de F_req (différence minimale)
diff = np.abs(Tz - F_req)
idx_zero_force = np.argmin(diff)
omega_zero_force = omega_rps[idx_zero_force]
puissance_zero_force = P_meca_arr[idx_zero_force]

# Dans le subplot puissance (3), on ajoutera :
plt.subplot(5,1,3)
plt.plot(omega_rps, P_meca_arr, 'r', label='Puissance mécanique (W)')
plt.plot(omega_rps, P_min_arr, 'g--', label='Puissance minimale drone trad (W)')

# Tracer un point sur la puissance à l'omega où somme forces proche de zéro
plt.plot(omega_zero_force, puissance_zero_force, 'bo', label=f'Puissance à somme forces =0 : {puissance_zero_force:.2f} W')

# Annotation texte près du point
plt.annotate(f'{puissance_zero_force:.2f} W',
            xy=(omega_zero_force, puissance_zero_force),
            xytext=(omega_zero_force+1, puissance_zero_force + 10),
            color='blue')

plt.ylabel('Puissance (W)')
plt.legend()
plt.grid()

plt.subplot(5,1,4)
plt.plot(omega_rps, P_perte_arr, 'm', label='Perte énergétique (W)')
plt.ylabel('Perte (W)')
plt.legend()
plt.grid()

plt.suptitle(f'Drone masse={m} kg, angle  $\alpha$ ={alpha_deg}° - Analyse puissance & pertes absolues')
plt.tight_layout(rect=[0, 0.03, 1, 0.97])
plt.show()

```

```

plt.figure(figsize=(8, 5))

# Filtrage : uniquement les points où  $\omega > 0$ 
omega_filtré = [w for w in omega_rps if w >= 9.55]
rendement_filtré = [r for w, r in zip(omega_rps, rendement_percent) if w >= 9.55]

plt.plot(omega_filtré, rendement_filtré, 'm-', linewidth=2, label='Rendement (%)')

plt.xlabel('Vitesse angulaire  $\omega$  (rad/s)')
plt.ylabel('Rendement (%)')
plt.title('Rendement énergétique en fonction de la vitesse angulaire ( $\omega \geq 9.55$  rad/s)')
plt.grid(True)
plt.legend()

# Texte tous les 50 points filtrés (ou moins s'il y en a peu)
for i in range(0, len(omega_filtré), 10):
    plt.text(omega_filtré[i], rendement_filtré[i], f"{rendement_filtré[i]:.1f}%", fontsize=8, ha='center', va='bottom', color='purple')

plt.show()

# Résumés console
if omega_Fprop_min is not None:
    print(f"Vitesse angulaire minimale pour Fprop  $\geq$  mg/sin( $\alpha$ ) : {omega_Fprop_min:.2f} tours/s")
else:
    print("Aucune vitesse suffisante pour la poussée totale requise.")

if omega_Tz_min is not None:
    print(f"Vitesse angulaire minimale pour Tz  $\geq$  mg : {omega_Tz_min:.2f} tours/s")
else:
    print("Aucune vitesse suffisante pour la poussée verticale requise.")

print(f"Puissance mécanique max = {np.max(P_meca_arr):.1f} W")
print(f"Perte énergétique max = {np.max(P_perte_arr):.1f} W")

delta_h_par_watt = []

# t = 1 # durée d'action (1 seconde)
# P_seuil = P_min_arr[0] # puissance juste suffisante pour flotter
#
# for i in range(len(omega)):
#     surplus_force = Tz[i] - m * g
#     surplus_puissance = P_meca_arr[i] - P_seuil
#     if surplus_puissance > 0:
#         # Travail utile = Force excédentaire * distance = m * g * delta_h

```

```

#         # Donc delta_h = Travail / (m * g) = (P * t) / (m * g)
#         delta_h = (surplus_force * t) / m # hauteur gagnée en 1 seconde
#         ratio = delta_h / surplus_puissance # m/W = gain par watt
#         delta_h_par_watt.append(ratio)
#     else:
#         delta_h_par_watt.append(0)
#
# plt.figure(figsize=(8, 5))
# plt.plot(omega_rps, delta_h_par_watt, 'b', label='Gain d\'altitude par Watt disponible (m/W)')
# plt.xlabel('Vitesse angulaire (tours/s)')
# plt.ylabel('Gain d\'altitude par Watt (m/W)')
# plt.title('Efficacité gravitaire de l\'énergie excédentaire')
# plt.grid(True)
# plt.legend()
# plt.show()
#

```

Drone sans ailes

```

import numpy as np
import matplotlib.pyplot as plt

# Constantes
rho = 1.225 # Masse volumique air (kg/m3)
r = 0.15 # Rayon hélice (m)
A = np.pi * r**2 # Surface disque propulsif (m²)
m = 1.5 # Masse drone (kg)
g = 9.81 # Gravité (m/s²)
alpha_deg = 12.5 # Angle d'inclinaison de la poussée (°)
alpha = np.radians(alpha_deg)

# Gamme vitesse angulaire (tours/s)
omega_rps = np.linspace(0.1, 45, 500) # tours/s
omega = omega_rps * 2 * np.pi # rad/s

# Hypothèses simplifiées : V0 et Ve proportionnels à omega * r
k1 = 0.3
k2 = 0.8

def vitesse_V0(omega):
    return k1 * omega * r

def vitesse_Ve(omega):
    return vitesse_V0(omega) + k2 * omega * r

```

```

def pousse(V_omega):
    V0 = vitesse_V0(V_omega)
    Ve = vitesse_Ve(V_omega)
    vi = (V0 + Ve) / 2
    return rho * A * vi * (Ve - V0), vi # poussée (N) et vitesse moyenne (m/s)

def puissance_mecanique(F_omega):
    Fprop, vi = pousse(F_omega)
    return Fprop * vi

def puissance_minimale(F_omega):
    # puissance minimale théorique pour le vol stationnaire
    return (m * g) ** 1.5 / np.sqrt(2 * rho * A)

def perte_energetique(F_omega):
    P_meca = puissance_mecanique(F_omega)
    P_min = puissance_minimale(F_omega)
    return P_meca - P_min # perte = puissance mécanique - puissance minimale

def calcul_puissances(F_omega):
    P_meca_list = []
    P_min_list = []
    rendement = []

```

```

def calcul_puissances(F_omega):
    P_meca_list = []
    P_min_list = []
    rendement = []

    for w in F_omega:
        P_meca = puissance_mecanique(w)
        P_min = puissance_minimale(w)
        P_min_list.append(P_min)
        P_meca_list.append(P_meca)
        # rendement en % (attention division par zéro possible)
        rdt = (1 - (P_min / P_meca)) * 100 if P_meca != 0 else 0
        rendement.append(rdt)

    return rendement

# Calcul du rendement en %
rendement_percent = calcul_puissances(omega)

# Initialisation des tableaux
Fprop_arr = np.zeros_like(omega)
vi_arr = np.zeros_like(omega)
P_meca_arr = np.zeros_like(omega)
P_min_arr = np.zeros_like(omega)
P_perte_arr = np.zeros_like(omega)

# Calculs pour chaque vitesse angulaire
for i, w in enumerate(omega):
    Fprop_arr[i], vi_arr[i] = poussee(w)
    P_meca_arr[i] = puissance_mecanique(w)
    P_min_arr[i] = puissance_minimale(w)
    P_perte_arr[i] = perte_energetique(w) # sans signe moins

# Poussée requise corrigée par l'angle
poids = m * g
F_req = poids / np.sin(alpha)

# Composante verticale de la poussée
Tz = Fprop_arr * np.sin(alpha)

# Recherche des vitesses angulaires minimales
idx_Fprop = np.where(Fprop_arr >= F_req)[0]
omega_Fprop_min = omega_rps[idx_Fprop[0]] if len(idx_Fprop) > 0 else None

idx_Tz = np.where(Tz >= poids)[0]

omega_Tz_min = omega_rps[idx_Tz[0]] if len(idx_Tz) > 0 else None

# Figure principale
plt.figure(figsize=(14, 12))

plt.subplot(5,1,1)
plt.plot(omega_rps, vitesse_V0(omega), label='Vitesse amont V0 (m/s)')
plt.plot(omega_rps, vitesse_Ve(omega), label='Vitesse sortie Ve (m/s)')
plt.ylabel('Vitesse (m/s)')
plt.legend()
plt.grid()

```

```

plt.subplot(5,1,2)
plt.plot(omega_rps, Fprop_arr, label='Poussée totale Fprop (N)')
plt.plot(omega_rps, np.full_like(omega_rps, F_req), 'k--', label='Poussée requise (mg / sin( $\alpha$ ))')
if omega_Fprop_min is not None:
    plt.axvline(omega_Fprop_min, color='r', linestyle=':', label=f'Q min poussée totale = {omega_Fprop_min:.2f} tours/s')
plt.ylabel('Force (N)')
plt.legend()
plt.grid()

# Point à 36.6 tours/s
omega_cible = 36.6
idx_36_6 = np.argmin(np.abs(omega_rps - omega_cible))
omega_point = omega_rps[idx_36_6]
puissance_point = P_meca_arr[idx_36_6]

plt.subplot(5,1,3)
plt.plot(omega_rps, P_meca_arr, 'r', label='Puissance mécanique (W)')
plt.plot(omega_rps, P_min_arr, 'g--', label='Puissance minimale drone trad (W)')
plt.plot(omega_point, puissance_point, 'bo', label=f'Puissance à 36.6 tours/s : {puissance_point:.2f} W')
plt.annotate(f'{puissance_point:.2f} W',
             xy=(omega_point, puissance_point),
             xytext=(omega_point + 1, puissance_point + 10),
             color='blue')
plt.ylabel('Puissance (W)')
plt.legend()
plt.grid()

plt.subplot(5,1,4)
plt.plot(omega_rps, P_perte_arr, 'm', label='Perte énergétique (W)')
plt.ylabel('Perte (W)')
plt.legend()
plt.grid()

plt.subplot(5,1,5)
plt.plot(omega_rps, Tz, label='Composante verticale Tz = Fprop*sin( $\alpha$ )')
plt.plot(omega_rps, np.full_like(omega_rps, poids), 'k--', label='Poids mg')
if omega_Tz_min is not None:
    plt.axvline(omega_Tz_min, color='r', linestyle=':', label=f'Q min poussée verticale = {omega_Tz_min:.2f} tours/s')
plt.xlabel('Vitesse angulaire (tours/s)')
plt.ylabel('Force verticale (N)')
plt.legend()
plt.grid()

plt.suptitle(f'Drone masse={m} kg, angle  $\alpha$ ={alpha_deg}° - Analyse puissance & pertes absolues')
plt.tight_layout(rect=[0, 0.03, 1, 0.97])

# Figure rendement uniquement pour  $\omega > 36.36$  tours/s
nlt.figure(figsize=(8, 5))

```

```

# Figure rendement uniquement pour  $\omega > 36.36$  tours/s
plt.figure(figsize=(8, 5))

omega_filtre = omega_rps[omega_rps > 36.36]
rendement_filtre = [r for w, r in zip(omega_rps, rendement_percent) if w > 36.36]

plt.plot(omega_filtre, rendement_filtre, 'm-', linewidth=2, label='Rendement (%)')

plt.xlabel('Vitesse angulaire  $\omega$  (tours/s)')
plt.ylabel('Rendement (%)')
plt.title('Rendement énergétique en fonction de la vitesse angulaire ( $\omega > 36.36$  tours/s)')
plt.grid(True)
plt.legend()

# Texte tous les 10 points (pour lisibilité)
for i in range(0, len(omega_filtre), 10):
    plt.text(omega_filtre[i], rendement_filtre[i], f"{rendement_filtre[i]:.1f}%", fontsize=8, ha='center', va='bottom', color='purple')

plt.show()

# Résumé console
if omega_Fprop_min is not None:
    print(f"Vitesse angulaire minimale pour  $F_{prop} \geq mg/\sin(\alpha)$  : {omega_Fprop_min:.2f} tours/s")
else:
    print("Aucune vitesse suffisante pour la poussée totale requise.")

if omega_Tz_min is not None:
    print(f"Vitesse angulaire minimale pour  $T_z \geq mg$  : {omega_Tz_min:.2f} tours/s")
else:
    print("Aucune vitesse suffisante pour la poussée verticale requise.")

print(f"Puissance mécanique max = {np.max(P_meca_arr):.1f} W")

print(f"Perte énergétique max = {np.max(P_perte_arr):.1f} W")

```

Coefficient de portance en fonction de la vitesse et de l'angle

```
import numpy as np
import matplotlib.pyplot as plt

# Constantes physiques et géométriques
masse = 1.5 # en kg
g = 9.81 # m/s²
rho = 1.225 # kg/m³ (air à 15°C)
S = 0.3 # Surface alaire en m²

# Plages de variations
vites = np.linspace(1, 30, 100) # vitesses de 1 à 30 m/s
angles_deg = np.linspace(0, 90, 100) # angles de 0° à 90°
angles_rad = np.radians(angles_deg)

# 1. CL en fonction de la vitesse pour différents angles
plt.figure(figsize=(10, 5))
for angle_deg in [0, 15, 30, 45, 60, 75, 90]:
    theta = np.radians(angle_deg)
    CL = (masse * g * np.cos(theta)) / (2 * rho * S * vites**2)
    plt.plot(vites, CL, label=f"{angle_deg}°")

plt.title("Coefficient de portance (CL) en fonction de la vitesse")
plt.xlabel("Vitesse (m/s)")
plt.ylabel("C_L")
plt.grid(True)
plt.legend(title="Angle (°)")
plt.tight_layout()
plt.show()

# 2. CL en fonction de l'angle pour différentes vitesses
plt.figure(figsize=(10, 5))
for V in [5, 10, 15, 20, 25]:
    CL = (masse * g * np.cos(angles_rad)) / (2 * rho * S * V**2)
    plt.plot(angles_deg, CL, label=f"{V} m/s")

plt.title("Coefficient de portance (CL) en fonction de l'angle")
plt.xlabel("Angle (°)")
plt.ylabel("C_L")
plt.grid(True)
plt.legend(title="Vitesse")
plt.tight_layout()
plt.show()
```

Loi coefficients

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import UnivariateSpline

# Données
Cp = np.array([0, 0.06, 0.12, 0.18, 0.24, 0.30, 0.36, 0.41, 0.47, 0.53, 0.58, 0.64, 0.69, 0.75, 0.80, 0.85, 0.91, 0.96, 1.01, 1.06, 1.11, 1.16, 1.20, 1.25, 1.2])
Ct = np.array([0.017, 0.018, 0.019, 0.021, 0.024, 0.027, 0.031, 0.035, 0.040, 0.046, 0.052, 0.058, 0.065, 0.073, 0.082, 0.091, 0.100, 0.111, 0.122, 0.133, 0.14])
angles = np.linspace(0, 19.5, len(Cp))

# Ajustement  $C_t = Ct_0 + k * Cp^2$ 
coeffs = np.polyfit(Cp**2, Ct, deg=1)
k, Ct0 = coeffs
Ct_fit = Ct0 + k * Cp**2

# --- Graphique Cp vs Ct ---
plt.figure(figsize=(8, 5))
plt.plot(Ct, Cp, 'o', label="Données")
plt.plot(Ct_fit, Cp, '-.', label=f"Ajustement :  $C_t = \{Ct0:.3f\} + \{k:.3f\} * Cp^2$ ")
plt.ylabel("Coefficient de portance (Cp)")
plt.xlabel("Coefficient de trainée (Ct)")
plt.title("Relation Cp vs Ct")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Calcul de la finesse ---
finesse = Cp / Ct

# Interpolation spline pour la finesse
spline = UnivariateSpline(angles, finesse, s=0.1) # s = lissage
angles_fine = np.linspace(angles[0], angles[-1], 500)
finesse_pred = spline(angles_fine)

# --- Tracé de la finesse ---
plt.figure(figsize=(8, 5))
plt.plot(angles, finesse, 'ro', label="Finesse mesurée (Cp/Ct)")
plt.plot(angles_fine, finesse_pred, 'b-', label="Interpolation spline")
plt.title("Finesse aérodynamique en fonction de l'angle d'incidence")
plt.xlabel("Angle d'incidence (°)")
plt.ylabel("Finesse (Cp / Ct)")
plt.grid(True)
plt.legend()

# Marquage du maximum de finesse
idx_max = np.argmax(finesse)
angle_opt = angles[idx_max]
finesse_max = finesse[idx_max]
plt.plot(angle_opt, finesse_max, 'ko', label=f"Max à {angle_opt:.1f}° ({finesse_max:.2f})")
plt.legend()

plt.tight_layout()
plt.show()
```

Profil avec vecteur

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Coordonnées de l'aile
extrados_x = np.array([-0.514, -0.5075, -0.48825, -0.45675, -0.4135, -0.35925, -0.29525, -0.22275, -0.14325, -0.05875, 0.02875, 0.11675, 0.20275, 0.28325, 0.3555, 0.416, 0.461])
extrados_y = np.array([0, 0.0185, 0.03575, 0.051, 0.06325, 0.07175, 0.07625, 0.0765, 0.07275, 0.0655, 0.0555, 0.044, 0.032, 0.021, 0.012, 0.0055, 0.00175, 0.00])
intrados_x = np.array([-0.514, -0.5075, -0.48825, -0.45675, -0.4135, -0.35925, -0.29525, -0.22275, -0.14325, -0.05875, 0.02875, 0.11675, 0.20275, 0.28325, 0.3555, 0.416, 0.461])
intrados_y = np.array([0, -0.0185, -0.03575, -0.051, -0.06325, -0.07175, -0.07625, -0.0765, -0.07275, -0.0655, -0.0555, -0.044, -0.032, -0.021, -0.012, -0.0055, 0.00175, 0.00])

# 2. Calculer la vitesse relative autour de l'aile (simplification)
v_inf = 1.0 # Vitesse loin de l'aile (m/s)
rho = 1.225 # Densité de l'air (kg/m³)
p_inf = 101325 # Pression atmosphérique (Pa)

# Hypothèse : la vitesse locale dépend de la courbure du profil (simplification pédagogique)
# Ici, on suppose une variation liée à la coordonnée y : plus y est élevé, plus la vitesse est grande
v_extrados = v_inf * (1 + extrados_y) # Accélération sur l'extrados
v_intrados = v_inf * (1 + intrados_y) # Ralentissement (valeurs négatives de y)

# 3. Calculer la pression par Bernoulli
p_extrados = p_inf - 0.5 * rho * (v_extrados**2 - v_inf**2)
p_intrados = p_inf - 0.5 * rho * (v_intrados**2 - v_inf**2)

# 4. Préparation des vecteurs de flèches
dx = np.zeros_like(extrados_x)
dy_extrados = (p_extrados / np.max(p_extrados)) * 0.1 * (-1)
dy_intrados = (p_intrados / np.max(p_intrados)) * 0.1 * (-1)

# 5. Affichage
plt.figure(figsize=(12, 6))
plt.plot(extrados_x, extrados_y, color='blue', label="Extrados")
plt.plot(intrados_x, intrados_y, color='red', label="Intrados")

plt.quiver(extrados_x, extrados_y, dx, -dy_extrados, color='blue', scale=1, scale_units='xy', angles='xy', label="Pression Extrados")
plt.quiver(intrados_x, intrados_y, dx, dy_intrados, color='red', scale=1, scale_units='xy', angles='xy', label="Pression Intrados")

plt.title("Distribution de la pression sur le profil d'aile")
plt.xlabel("x (corde)")
plt.ylabel("y (épaisseur)")
plt.axis('equal')
plt.grid(True)
plt.legend()

# 6. Calcul de la portance (simplifiée)
delta_p = p_intrados - p_extrados
portance = np.trapezoid(delta_p, extrados_x)
print(f"Portance approximative (par mètre de profondeur) : {portance:.2f} N/m")
plt.show()
```

```

##Bibliotheques :
import numpy as np
import matplotlib.pyplot as plt

##Parametres :

mu = 1.8*10**(-5)      #Pascal-seconde  viscosité dynamique de l air
ro = 1.225              #kg/m3   densité de l air   ( ne sera pas une constante l avoir en fonction)
TemperatureC = 20      #degrés   (faire une fonction pour determiner ro en fonction de la temps)
g= 9.81                # accélération gravitationnel

altitude =0            #Altitude en m

#caractéristique du drone :

vitesse = 15           #vitesse de l aile ou de l air (m secondes)
vitesse_induite =0
hauteur_aile = 0.05    # metre
longueur_aile=0.2      # determiner L en metre ( 20 cm)  (rendre sa utile)
m=0.5                  #Masse du drone kg
n= 4                   #nombre d aile sur le drone
largeur_aile=0.15

#caracteristique des rotors du drones(  FIXES / bras )

coefficient_portance_petite_aile = 1.2 # Coefficient de portance (experimental)
p=2 #nombre de petite ailes par rotor
aire_aillaire_petite_aile = 0.0006    #metre carré
r=0.03

#calcul de l air déplacé par le drone ( cas d un drone usuel)

#Aspect Energétiques  (mecaniques , cintétique  et potentiel , puissance etc ) :

F_min=m*g              #Force de portance minimal necessaire au vol du drone  ( en gros just pfd en fonction du poids)

###listes

portances=[]
trainees=[]

```

```

##listes

portances=[]
trainees=[]

##Particularité de l aile
#ici aile symétrique
angles = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.5, 10.0, 10.5, 11.0, 11.5, 12.0, 12.5, 13.0, 13.5, 14.0]

# PORTANCE_solidworks=[0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.5, 10.0, 10.5, 11.0, 11.5, 12.0, 12.5, 13.0, 13.5, 14.0]
#
# TRAINER_solidworks=[0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.5, 10.0, 10.5, 11.0, 11.5, 12.0, 12.5, 13.0, 13.5, 14.0]
#

coefficient_portance = [0, 0.06, 0.12, 0.18, 0.24, 0.30, 0.36, 0.41, 0.47, 0.53, 0.58, 0.64, 0.69, 0.75, 0.80, 0.85, 0.91, 0.96, 1.01, 1.06, 1.11, 1.16, 1.20, 1.25, 1.30, 1.35, 1.40]
coefficient_trainee = [0.017,0.018,0.019,0.021,0.024,0.027,0.031,0.035,0.040,0.046,0.052,0.058,0.065,0.073,0.082,0.091,0.100,0.111,0.122,0.133,0.146,0.159,0.175,0.190,0.205,0.220,0.235,0.250,0.265,0.280,0.295,0.310,0.325,0.340,0.355,0.370,0.385,0.400,0.415,0.430,0.445,0.460,0.475,0.490,0.505,0.520,0.535,0.550,0.565,0.580,0.595,0.610,0.625,0.640,0.655,0.670,0.685,0.700,0.715,0.730,0.745,0.760,0.775,0.790,0.805,0.820,0.835,0.850,0.865,0.880,0.895,0.910,0.925,0.940,0.955,0.970,0.985,1.000,1.015,1.030,1.045,1.060,1.075,1.090,1.105,1.120,1.135,1.150,1.165,1.180,1.195,1.210,1.225,1.240,1.255,1.270,1.285,1.300,1.315,1.330,1.345,1.360,1.375,1.390,1.405,1.420,1.435,1.450,1.465,1.480,1.495,1.510,1.525,1.540,1.555,1.570,1.585,1.600,1.615,1.630,1.645,1.660,1.675,1.690,1.705,1.720,1.735,1.750,1.765,1.780,1.795,1.810,1.825,1.840,1.855,1.870,1.885,1.900,1.915,1.930,1.945,1.960,1.975,1.990,2.005,2.020,2.035,2.050,2.065,2.080,2.095,2.110,2.125,2.140,2.155,2.170,2.185,2.200,2.215,2.230,2.245,2.260,2.275,2.290,2.305,2.320,2.335,2.350,2.365,2.380,2.395,2.410,2.425,2.440,2.455,2.470,2.485,2.500,2.515,2.530,2.545,2.560,2.575,2.590,2.605,2.620,2.635,2.650,2.665,2.680,2.695,2.710,2.725,2.740,2.755,2.770,2.785,2.800,2.815,2.830,2.845,2.860,2.875,2.890,2.905,2.920,2.935,2.950,2.965,2.980,2.995,3.010,3.025,3.040,3.055,3.070,3.085,3.100,3.115,3.130,3.145,3.160,3.175,3.190,3.205,3.220,3.235,3.250,3.265,3.280,3.295,3.310,3.325,3.340,3.355,3.370,3.385,3.400,3.415,3.430,3.445,3.460,3.475,3.490,3.505,3.520,3.535,3.550,3.565,3.580,3.595,3.610,3.625,3.640,3.655,3.670,3.685,3.700,3.715,3.730,3.745,3.760,3.775,3.790,3.805,3.820,3.835,3.850,3.865,3.880,3.895,3.910,3.925,3.940,3.955,3.970,3.985,4.000,4.015,4.030,4.045,4.060,4.075,4.090,4.105,4.120,4.135,4.150,4.165,4.180,4.195,4.210,4.225,4.240,4.255,4.270,4.285,4.300,4.315,4.330,4.345,4.360,4.375,4.390,4.405,4.420,4.435,4.450,4.465,4.480,4.495,4.510,4.525,4.540,4.555,4.570,4.585,4.600,4.615,4.630,4.645,4.660,4.675,4.690,4.705,4.720,4.735,4.750,4.765,4.780,4.795,4.810,4.825,4.840,4.855,4.870,4.885,4.900,4.915,4.930,4.945,4.960,4.975,4.990,5.005,5.020,5.035,5.050,5.065,5.080,5.095,5.110,5.125,5.140,5.155,5.170,5.185,5.200,5.215,5.230,5.245,5.260,5.275,5.290,5.305,5.320,5.335,5.350,5.365,5.380,5.395,5.410,5.425,5.440,5.455,5.470,5.485,5.500,5.515,5.530,5.545,5.560,5.575,5.590,5.605,5.620,5.635,5.650,5.665,5.680,5.695,5.710,5.725,5.740,5.755,5.770,5.785,5.800,5.815,5.830,5.845,5.860,5.875,5.890,5.905,5.920,5.935,5.950,5.965,5.980,5.995,6.010,6.025,6.040,6.055,6.070,6.085,6.100,6.115,6.130,6.145,6.160,6.175,6.190,6.205,6.220,6.235,6.250,6.265,6.280,6.295,6.310,6.325,6.340,6.355,6.370,6.385,6.400,6.415,6.430,6.445,6.460,6.475,6.490,6.505,6.520,6.535,6.550,6.565,6.580,6.595,6.610,6.625,6.640,6.655,6.670,6.685,6.700,6.715,6.730,6.745,6.760,6.775,6.790,6.805,6.820,6.835,6.850,6.865,6.880,6.895,6.910,6.925,6.940,6.955,6.970,6.985,7.000,7.015,7.030,7.045,7.060,7.075,7.090,7.105,7.120,7.135,7.150,7.165,7.180,7.195,7.210,7.225,7.240,7.255,7.270,7.285,7.300,7.315,7.330,7.345,7.360,7.375,7.390,7.405,7.420,7.435,7.450,7.465,7.480,7.495,7.510,7.525,7.540,7.555,7.570,7.585,7.600,7.615,7.630,7.645,7.660,7.675,7.690,7.705,7.720,7.735,7.750,7.765,7.780,7.795,7.810,7.825,7.840,7.855,7.870,7.885,7.900,7.915,7.930,7.945,7.960,7.975,7.990,8.005,8.020,8.035,8.050,8.065,8.080,8.095,8.110,8.125,8.140,8.155,8.170,8.185,8.200,8.215,8.230,8.245,8.260,8.275,8.290,8.305,8.320,8.335,8.350,8.365,8.380,8.395,8.410,8.425,8.440,8.455,8.470,8.485,8.500,8.515,8.530,8.545,8.560,8.575,8.590,8.605,8.620,8.635,8.650,8.665,8.680,8.695,8.710,8.725,8.740,8.755,8.770,8.785,8.800,8.815,8.830,8.845,8.860,8.875,8.890,8.905,8.920,8.935,8.950,8.965,8.980,8.995,9.010,9.025,9.040,9.055,9.070,9.085,9.100,9.115,9.130,9.145,9.160,9.175,9.190,9.205,9.220,9.235,9.250,9.265,9.280,9.295,9.310,9.325,9.340,9.355,9.370,9.385,9.400,9.415,9.430,9.445,9.460,9.475,9.490,9.505,9.520,9.535,9.550,9.565,9.580,9.595,9.610,9.625,9.640,9.655,9.670,9.685,9.700,9.715,9.730,9.745,9.760,9.775,9.790,9.805,9.820,9.835,9.850,9.865,9.880,9.895,9.910,9.925,9.940,9.955,9.970,9.985,10.000,10.015,10.030,10.045,10.060,10.075,10.090,10.105,10.120,10.135,10.150,10.165,10.180,10.195,10.210,10.225,10.240,10.255,10.270,10.285,10.300,10.315,10.330,10.345,10.360,10.375,10.390,10.405,10.420,10.435,10.450,10.465,10.480,10.495,10.510,10.525,10.540,10.555,10.570,10.585,10.600,10.615,10.630,10.645,10.660,10.675,10.690,10.705,10.720,10.735,10.750,10.765,10.780,10.795,10.810,10.825,10.840,10.855,10.870,10.885,10.900,10.915,10.930,10.945,10.960,10.975,10.990,11.005,11.020,11.035,11.050,11.065,11.080,11.095,11.110,11.125,11.140,11.155,11.170,11.185,11.200,11.215,11.230,11.245,11.260,11.275,11.290,11.305,11.320,11.335,11.350,11.365,11.380,11.395,11.410,11.425,11.440,11.455,11.470,11.485,11.500,11.515,11.530,11.545,11.560,11.575,11.590,11.605,11.620,11.635,11.650,11.665,11.680,11.695,11.710,11.725,11.740,11.755,11.770,11.785,11.800,11.815,11.830,11.845,11.860,11.875,11.890,11.905,11.920,11.935,11.950,11.965,11.980,11.995,12.010,12.025,12.040,12.055,12.070,12.085,12.100,12.115,12.130,12.145,12.160,12.175,12.190,12.205,12.220,12.235,12.250,12.265,12.280,12.295,12.310,12.325,12.340,12.355,12.370,12.385,12.400,12.415,12.430,12.445,12.460,12.475,12.490,12.505,12.520,12.535,12.550,12.565,12.580,12.595,12.610,12.625,12.640,12.655,12.670,12.685,12.700,12.715,12.730,12.745,12.760,12.775,12.790,12.805,12.820,12.835,12.850,12.865,12.880,12.895,12.910,12.925,12.940,12.955,12.970,12.985,13.000,13.015,13.030,13.045,13.060,13.075,13.090,13.105,13.120,13.135,13.150,13.165,13.180,13.195,13.210,13.225,13.240,13.255,13.270,13.285,13.300,13.315,13.330,13.345,13.360,13.375,13.390,13.405,13.420,13.435,13.450,13.465,13.480,13.495,13.510,13.525,13.540,13.555,13.570,13.585,13.600,13.615,13.630,13.645,13.660,13.675,13.690,13.705,13.720,13.735,13.750,13.765,13.780,13.795,13.810,13.825,13.840,13.855,13.870,13.885,13.900,13.915,13.930,13.945,13.960,13.975,13.990,14.005,14.020,14.035,14.050,14.065,14.080,14.095,14.110,14.125,14.140,14.155,14.170,14.185,14.200,14.215,14.230,14.245,14.260,14.275,14.290,14.305,14.320,14.335,14.350,14.365,14.380,14.395,14.410,14.425,14.440,14.455,14.470,14.485,14.500,14.515,14.530,14.545,14.560,14.575,14.590,14.605,14.620,14.635,14.650,14.665,14.680,14.695,14.710,14.725,14.740,14.755,14.770,14.785,14.800,14.815,14.830,14.845,14.860,14.875,14.890,14.905,14.920,14.935,14.950,14.965,14.980,14.995,15.010,15.025,15.040,15.055,15.070,15.085,15.100,15.115,15.130,15.145,15.160,15.175,15.190,15.205,15.220,15.235,15.250,15.265,15.280,15.295,15.310,15.325,15.340,15.355,15.370,15.385,15.400,15.415,15.430,15.445,15.460,15.475,15.490,15.505,15.520,15.535,15.550,15.565,15.580,15.595,15.610,15.625,15.640,15.655,15.670,15.685,15.700,15.715,15.730,15.745,15.760,15.775,15.790,15.805,15.820,15.835,15.850,15.865,15.880,15.895,15.910,15.925,15.940,15.955,15.970,15.985,16.000,16.015,16.030,16.045,16.060,16.075,16.090,16.105,16.120,16.135,16.150,16.165,16.180,16.195,16.210,16.225,16.240,16.255,16.270,16.285,16.300,16.315,16.330,16.345,16.360,16.375,16.390,16.405,16.420,16.435,16.450,16.465,16.480,16.495,16.510,16.525,16.540,16.555,16.570,16.585,16.600,16.615,16.630,16.645,16.660,16.675,16.690,16.705,16.720,16.735,16.750,16.765,16.780,16.795,16.810,16.825,16.840,16.855,16.870,16.885,16.900,16.915,16.930,16.945,16.960,16.975,16.990,17.005,17.020,17.035,17.050,17.065,17.080,17.095,17.110,17.125,17.140,17.155,17.170,17.185,17.200,17.215,17.230,17.245,17.260,17.275,17.290,17.305,17.320,17.335,17.350,17.365,17.380,17.395,17.410,17.425,17.440,17.455,17.470,17.485,17.500,17.515,17.530,17.545,17.560,17.575,17.590,17.605,17.620,17.635,17.650,17.665,17.680,17.695,17.710,17.725,17.740,17.755,17.770,17.785,17.800,17.815,17.830,17.845,17.860,17.875,17.890,17.905,17.920,17.935,17.950,17.965,17.980,17.995,18.010,18.025,18.040,18.055,18.070,18.085,18.100,18.115,18.130,18.145,18.160,18.175,18.190,18.205,18.220,18.235,18.250,18.265,18.280,18.295,18.310,18.325,18.340,18.355,18.370,18.385,18.400,18.415,18.430,18.445,18.460,18.475,18.490,18.505,18.520,18.535,18.550,18.565,18.580,18.595,18.610,18.625,18.640,18.655,18.670,18.685,18.700,18.715,18.730,18.745,18.760,18.775,18.790,18.805,18.820,18.835,18.850,18.865,18.880,18.895,18.910,18.925,18.940,18.955,18.970,18.985,19.000,19.015,19.030,19.045,19.060,19.075,19.090,19.105,19.120,19.135,19.150,19.165,19.180,19.195,19.210,19.225,19.240,19.255,19.270,19.285,19.300,19.315,19.330,19.345,19.360,19.375,19.390,19.405,19.420,19.435,19.450,19.465,19.480,19.495,19.510,19.525,19.540,19.555,19.570,19.585,19.600,19.615,19.630,19.645,19.660,19.675,19.690,19.705,19.720,19.735,19.750,19.765,19.780,19.795,19.810,19.825,19.840,19.855,19.870,19.885,19.900,19.915,19.930,19.945,19.960,19.975,19.990,20.005,20.020,20.035,20.050,20.065,20.080,20.095,20.110,20.125,20.140,20.155,20.170,20.185,20.200,20.215,20.230,20.245,20.260,20.275,20.290,20.305,20.320,20.335,20.350,20.365,20.380,20.395,20.410,20.425,20.440,20.455,20.470,20.485,20.500,20.515,20.530,20.545,20.560,20.575,20.590,20.605,20.620,20.635,20.650,20.665,20.680,20.695,20.710,20.725,20.740,20.755,20.770,20.785,20.800,20.815,20.830,20.845,20.860,20.875,20.890,20.905,20.920,20.935,20.950,20.965,20.980,20.995,21.010,21.025,21.040,21.055,21.070,21.085,21.100,21.115,21.130,21.145,21.160,21.175,21.190,21.205,21.220,21.235,21.250,21.265,21.280,21.295,21.310,21.325,21.340,21.355,21.370,21.385,21.400,21.415,21.430,21.445,21.460,21.475,21.490,21.505,21.520,21.535,21.550,21.565,21.580,21.595,21.610,21.625,21.640,21.655,21.670,21.685,21.700,21.715,21.730,21.745,21.760,21.775,21.790,21.805,21.820,21.835,21.850,21.865,21.880,21.895,21.910,21.925,21.940,21.955,21.970,21.985,22.000,22.015,22.030,22.045,22.060,22.075,22.090,22.105,22.120,22.135,22.150,22.165,22.180,22.195,22.210,22.225,22.240,22.255,22.270,22.285,22.300,22.315,22.330,22.345,22.360,22.375,22.390,22.405,22.420,22.435,22.450,22.465,22.480,22.495,22.510,22.525,22.540,22.555,22.570,22.585,22.600,22.615,22.630,22.645,22.660,22.675,22.690,22.705,22.720,22.735,22.750,22.765,22.780,22.795,22.810,22.825,22.840,22.855,22.870,22.885,22.900,22.915,22.930,22.945,22.960,22.975,22.990,23.005,23.020,23.035,23.050,23.065,23.080,23.095,23.110,23.125,23.140,23.155,23.170,23.185,23.200,23.215,23.230,23.245,23.260,23.275,23.290,23.305,23.320,23.335,23.350,23.365,23.380,23.395,23.410,23.425,23.440,23.455,23.470,23.485,23.500,23.515,23.530,23.545,23.560,23.575,23.590,23.605,23.620,23.635,23.650,23.665,23.680,23.695,23.710,23.725,23.740,23.755,23.770,23.785,23.800,23.815,23.830,23.845,23.860,23.875,23.890,23.905,23.920,23.935,23.950,23.965,23.980,23.995,24.010,24.025,24.040,24.055,24.070,24.085,24.100,24.115,24.130,24.145,24.160,24.175,24.190,24.205,24.220,24.235,24.250,24.265,24.280,24.295,24.310,24.325,24.340,24.355,24.370,24.385,24.400,24.415,24.430,24.445,24.460,24.475,24.490,24.505,24.520,24.535,24.550,24.565,24.580,24.595,24.610,24.625,24.640,24.655,24.670,24.685,24.700,24.715,24.730,24.745,24.760,24.775,24.790,24.805,24.820,24.835,24.850,24.865,24.880,24.895,24.910,24.925,24.940,24.955,24.970,24.985,25.000,25.015,25.030,25.045,25.060,25.075,25.090,25.105,25.120,25.135,25.150,25.165,25.180,25.195,25.210,25.225,25.240,25.255,25.270,25.285,25.300,25.315,25.330,25.345,25.360,25.375,25.390,25.405,25.420,25.435,25.450,25.465,25.480,25.495,25.510,25.525,25.540,25.555,25.570,25.585,25.600,25.615,25.630,25.645,25.660,25.675,25.690,25.705,25.720,25.735,25.750,25.765,25.780,25.795,25.810,25.825,25.840,25.855,25.870,25.885,25.900,25.915,25.930,25.945,25.960,25.975,25.990,26.005,26.020,26.035,26.050,26.065,26.080,26.095,26.110,26.125,26.140,26.155,26.170,26.185,26.200,26.215,26.230,26.245,26.260,26.275,26.290,26
```

```

aire_interne = 0
for i in range(len(x_int) - 1):
    aire_interne += 0.5 * (x_int[i + 1] - x_int[i]) * (y_ext[i] + y_ext[i + 1] - y_int[i] - y_int[i + 1])
return abs(aire_interne)

def calcul_aire_aillage(x, y):
    aire_aillage = 0
    for i in range(len(x) - 1):
        aire_aillage += 0.5 * (x[i + 1] - x[i]) * (y[i] + y[i + 1]) # Somme des trapèzes
    return abs(aire_aillage)

## Formules

perimetre = calcul_perimetre(extrados_x, extrados_y)

Aire_interne = calcul_aire_interne(intrados_x, intrados_y, extrados_x, extrados_y)

Surface_frontale = perimetre * longueur_aile # c est pour 1 SEUL aile (oui car plan * longueur (axe y) = 3Dim)

print("la surface frontale de 1 aile est de:", Surface_frontale,"m^2","pour une longueur de 1 aile selon x de :",perimetre,"m" )

## Calcul des portances du drones ( creer)

for cp, ct in zip(coefficient_portance, coefficient_trainee):
    portance = cp * 0.5 * ro * (vitesse**2) * Surface_frontale
    trainee = ct * 0.5 * ro * (vitesse**2) * Surface_frontale
    portances.append(portance)
    trainees.append(trainee)

# Trouver la valeur maximale de la portance et l'angle correspondant avec la portance a cet angles
max_portance = max(portances)

max_index = portances.index(max_portance)
angle_max_portance = angles[max_index]
trainee_max_portance = round(trainees[max_index],2)
force_max_utile=round(max_portance-trainee_max_portance,3)

cp_angle15_aile= coefficient_portance[max_index]

# Afficher uniquement le coefficient de portance correspondant
coefficient_portance[max_index]

print(max_portance,max_index,angle_max_portance,trainee_max_portance,force_max_utile)

## Histogrammes
bar_width = 0.5
indices = np.arange(len(angles))
plt.figure(figsize=(12, 6))

# Histogramme pour la portance
plt.bar(indices, portances, bar_width, label='Portance', color='blue')

# Histogramme pour la trainée, légèrement décalé pour qu'ils ne se superposent pas
plt.bar(indices + bar_width, trainees, bar_width, label='Trainée', color='red')
plt.axvline(max_index, color='green', linestyle='--', label=f"Portance Max à {angle_max_portance}°")

```

```

print(max_portance,max_index,angle_max_portance,trainee_max_portance,force_max_utile)

## histogrammes
bar_width = 0.5
indices = np.arange(len(angles))
plt.figure(figsize=(12, 6))

# Histogramme pour la portance
plt.bar(indices, portances, bar_width, label='Portance', color='blue')

# Histogramme pour la trainée, légèrement décalé pour qu'ils ne se superposent pas
plt.bar(indices + bar_width, trainees, bar_width, label='Trainée', color='red')
plt.axvline(max_index, color='green', linestyle='--', label=f'Portance Max à {angle_max_portance}°')

##COMPARAISON des modeles ( traditionnel a ceux crée)
#( on regarde la puissance nécessaire pour etre l equilibre par une aile)

#angle désiré:      1rad * 180/π = 57,296°
                   # 0.0174533 rad          = 1°

angle_désiré_en_radian = 0.0174533 * (90- angle_max_portance)    # on veut 15 degré ici en radian
portance_bras_ailé = max_portance

##### je crois avoir calculer la puissance produite et non la puissance necessaire EFFECTIVEMENT EREUKA

#Drone traditionnel a 4 moteur :

vitesse_air_equilibre_trad = ( (m*g*2)/(ro* aire_aillaire_petite_aile *coefficient_portance_petite_aile) )**(1/2)
Portance_petite_aile_equilibre = (m*g)                                #portance pour une unique aile (sur un total de 4)
Puissance_traditionnel = Portance_petite_aile_equilibre * vitesse_air_equilibre_trad
print(*vitesse_trad *,vitesse_air_equilibre_trad)

#oui c est correct pour le 1 er modele .

#Drone SANS bras aillés a 15 ° :

vitesse air equilibre angle = ( (m*g*2)/(ro* aire_aillaire_petite_aile*p *coefficient_portance_petite_aile*np.cos(angle_désiré_en_radian)) )**(1/2)

print(*vitesse_sans *,vitesse_air_equilibre_angle)

Puissance_Sans_bras_ailé = ((m*g) *np.cos(angle_désiré_en_radian) *n * vitesse_air_equilibre_angle    # a verif

#Drone AVBC bras aillés a 15 ° :
```

```

vitesse_air_equilibre_angle_AVEC_ailes = { (m*g*2)/(ro* (aire_ailaire_petite_aile*p*(coefficient_portance_petite_aile) + (Surface_frontale* cp_angle15_aile) )

#on suppose que vitesse de la air est uniforme sur tout le bras (aile + rotor)

print("vitesse avec ",vitesse_air_equilibre_angle_AVEC_ailes)

#ici on a la puissance produite
Puissance_Avec_bras_ailé = portance_bras_ailé*n* vitesse_air_equilibre_angle_AVEC_ailes # VRAI TXT

Puissance_Avec_bras_ailé_rotor = p*n* ro* aire_ailaire_petite_aile *(vitesse_air_equilibre_angle_AVEC_ailes**2)*coefficient_portance_petite_aile

PUISSANCE_Théorique = portance_bras_ailé *vitesse *n

def vitesse_radiale(v_m_s, rayon):
    # Conversion de la vitesse linéaire à une vitesse angulaire en radians par seconde
    omega = v_m_s / rayon #  $\omega = v / r$ 
    return (omega)

print(
    "trad:", round(vitesse_radiale(vitesse_air_equilibre_trad, 0.03), 2),
    "sans ail:", round(vitesse_radiale(vitesse_air_equilibre_angle, 0.03), 2),
    "avec aile :", round(vitesse_radiale(vitesse_air_equilibre_angle_AVEC_ailes, 0.03), 2) ,
    "en rad/seconde :!"
)

##Print généraux

print("=====ENERGIE NECESSAIRE AU VOL Des DRONES PAR AILES a",angle_max_portance ,"" , "=====")

print("puissance minimal électrique au vol d'un drone traditionnel: " ,round(Puissance_traditionnel,2),"Watts")

print("puissance minimal électrique au vol d'un drone SANS bras ailé :",round(Puissance_Sans_bras_ailé,2),"Watts")

print("puissance minimal électrique au vol d'un drone AVEC bras ailés: " ,round(Puissance_Avec_bras_ailé_rotor,2)
,"Watts")

print("puissance total produite électrique au vol d'un drone AVEC bras ailés: " ,round(Puissance_Avec_bras_ailé,2)
,"Watts")

print("=====")

print("la portance maximal est de " , round(max_portance,3),"Newtons","pour un angle max",angle_max_portance , "et une valeur de traînée de" ,round(traînée_max_p
,soit un force max utile de" ,force_max_utile,"N" )

print(" ")
print("Puissance produite : " ,PUISSANCE_Théorique,"W")

##test

#Annotation sur le graphique des portances et de la traînée
point_a_annoter = list(range(0, len(angles), 1))
angles_np = np.array(angles) # Conversion en tableau NumPy
portances_np = np.array(portances)
traînees_np = np.array(traînees)
# Points sur les barres pour la portance et la traînée
plt.scatter(indices, portances_np, zorder=5)
plt.scatter(indices + bar_width, traînees_np, zorder=5)

# Ajouter des annotations sur chaque point AVEC un décalage en escalier
for i in range(len(angles_np)):
    if i % 3 == 0:
        offset = -10
    elif i % 3 == 1:
        offset = 5
    else:
        offset = +15 # décalage alternatif pour éviter le chevauchement
    plt.annotate(f'({portances_np[i]:.1f})',
        (indices[i], portances_np[i]),
        textcoords="offset points", xytext=(0, offset), ha="center", fontsize=8, color="orange", zorder=10)
    plt.annotate(f'({traînees_np[i]:.1f})',
        (indices[i] + bar_width, traînees_np[i]),
        textcoords="offset points", xytext=(0, offset), ha="center", fontsize=8, color="cyan", zorder=10)

```

```

# Labels et titre de l histogramme
plt.xlabel('Angle d\'attaque (degrés)')
plt.ylabel('Force (N)')
plt.title('Histogramme des forces de Portance et de Trainée en fonction de l\'angle d\'attaque')
plt.xticks(indices + bar_width / 2, angles, rotation=45)
plt.legend()
plt.grid(True)

# Affichage

##courbes 2
# Affichage des résultats

#print("Portances:", portances)
#print("Trainées:", trainees)

plt.figure(figsize=(9, 6))

plt.plot(angles, portances, label='Portance', color='blue')
plt.plot(angles, trainees, label='Trainée', color='red')
plt.xlabel('Angle d\'attaque (degrés)')
plt.ylabel('Force (N)')
plt.title('Portance et Trainée en fonction de l\'angle d\'attaque')
plt.legend()
plt.grid(True)

##

coeff_PORTANCE_solidworks=[]
coeff_TRAINEE_solidworks=[]
#calculer la portance et trainée obtenue puis comparer via le reel , ecrire egalement la valeur de rendement { exp par rapp au theorique }

for i in range(len(angles)):
    # Calcul du coefficient de portance
    cp = float(PORTANCE_solidworks[i] / (0.5 * ro * (vitesse**2) * Surface_frontale) )
    cp=round(cp,4)
    coeff_PORTANCE_solidworks.append(cp)

    # Calcul du coefficient de trainée (à définir selon vos besoins)
    cd = float(TRAINER_solidworks[i] / (0.5 * ro * (vitesse**2) * Surface_frontale) )
    cd =round(cd,4)
    coeff_TRAINEE_solidworks.append(cd)

# Résultat final
print("Coeff de portance issue de solidworks:", coeff_PORTANCE_solidworks)
print(" ")
print("Coeff de trainée issue de solidworks :", coeff_TRAINEE_solidworks)
print(" ")

##
#courbe
plt.figure(figsize=(5, 5))
plt.plot(extrados_x, extrados_y, label='Extrados', color='black')
plt.plot(intrados_x, intrados_y, label='Intrados', color='blue')

#Séparer les vecteurs en fonction des signes des composantes (Pression vectorielle )

for i in range(len(pression_extrados)):
    plt.quiver(extrados_x[i], extrados_y[i], pression_extrados[i], extrados_x[i], color='purple', angles='xy', scale_units='xy', scale=(1500))

for i in range(len(pression_intrados)):
    plt.quiver(intrados_x[i], intrados_y[i], pression_intrados[i], intrados_x[i], color='orange', angles='xy', scale_units='xy', scale=(1500))

plt.title('Cartographie des pression autour de l aile ')
plt.xlabel('x')
plt.ylabel('y')
plt.grid(True)
plt.axis('equal')
plt.legend()
plt.show()

```

```

##Bibliotheques :
import numpy as np
import matplotlib.pyplot as plt

##Parametres :

mu = 1.8*10**(-5)      #Pascal-seconde viscosité dynamique de l air
ro = 1.225              #kg/m3 densité de l air
TemperatureC = 20      #degrés
g= 9.81                # accélération gravitationnel
altitude =0            #Altitude en m

#caractéristique du drone :

vitesse = 25           #vitesse de l aile ou de l air (m /secondes)
vitesse_induite =0
hauteur_aile = 0.05    # metre
longueur_aile=0.2      # determiner L en metre ( 20 cm)
m=0.5                  #Masse du drone kg
n= 4                   #nombre d aile sur le drone

#caracteristique des rotors du drones(  FIXES / bras )

coefficient_portance_petite_aile = 1.2 # Coefficient de portance (experimental)
p=2 #nombre de petite ailes par rotor
aire_ailaire_petite_aile = 0.0006      #metre carré
r=0.2 #rayon bras
Coeff_port_rotor = 1.2 # carracteristique des drones RC .

##listes

portances=[]
trainees=[]

##Particularité de l aile

#Aile symétrique :

angles = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0, 5.5, 6.0, 6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 9.5, 10.0, 10.5, 11.0, 11.5, 12.0, 12.5, 13.0, 13.5, 14.0, 14.5, 15.0, 15.5, 16.0, 16.5, 17.0, 17.5, 18.0, 18.5, 19.0, 19.5, 20.0]

coefficient_portance = [0, 0.06, 0.12, 0.18, 0.24, 0.30, 0.36, 0.41, 0.47, 0.53, 0.58, 0.64, 0.69, 0.75, 0.80, 0.85, 0.91, 0.96, 1.01, 1.06, 1.11, 1.16, 1.20, 1.25, 1.30, 1.35, 1.40, 1.45, 1.50, 1.55, 1.60, 1.65, 1.70, 1.75, 1.80, 1.85, 1.90, 1.95, 2.00, 2.05, 2.10, 2.15, 2.20, 2.25, 2.30, 2.35, 2.40, 2.45, 2.50, 2.55, 2.60, 2.65, 2.70, 2.75, 2.80, 2.85, 2.90, 2.95, 3.00, 3.05, 3.10, 3.15, 3.20, 3.25, 3.30, 3.35, 3.40, 3.45, 3.50, 3.55, 3.60, 3.65, 3.70, 3.75, 3.80, 3.85, 3.90, 3.95, 4.00, 4.05, 4.10, 4.15, 4.20, 4.25, 4.30, 4.35, 4.40, 4.45, 4.50, 4.55, 4.60, 4.65, 4.70, 4.75, 4.80, 4.85, 4.90, 4.95, 5.00, 5.05, 5.10, 5.15, 5.20, 5.25, 5.30, 5.35, 5.40, 5.45, 5.50, 5.55, 5.60, 5.65, 5.70, 5.75, 5.80, 5.85, 5.90, 5.95, 6.00, 6.05, 6.10, 6.15, 6.20, 6.25, 6.30, 6.35, 6.40, 6.45, 6.50, 6.55, 6.60, 6.65, 6.70, 6.75, 6.80, 6.85, 6.90, 6.95, 7.00, 7.05, 7.10, 7.15, 7.20, 7.25, 7.30, 7.35, 7.40, 7.45, 7.50, 7.55, 7.60, 7.65, 7.70, 7.75, 7.80, 7.85, 7.90, 7.95, 8.00, 8.05, 8.10, 8.15, 8.20, 8.25, 8.30, 8.35, 8.40, 8.45, 8.50, 8.55, 8.60, 8.65, 8.70, 8.75, 8.80, 8.85, 8.90, 8.95, 9.00, 9.05, 9.10, 9.15, 9.20, 9.25, 9.30, 9.35, 9.40, 9.45, 9.50, 9.55, 9.60, 9.65, 9.70, 9.75, 9.80, 9.85, 9.90, 9.95, 10.00, 10.05, 10.10, 10.15, 10.20, 10.25, 10.30, 10.35, 10.40, 10.45, 10.50, 10.55, 10.60, 10.65, 10.70, 10.75, 10.80, 10.85, 10.90, 10.95, 11.00, 11.05, 11.10, 11.15, 11.20, 11.25, 11.30, 11.35, 11.40, 11.45, 11.50, 11.55, 11.60, 11.65, 11.70, 11.75, 11.80, 11.85, 11.90, 11.95, 12.00, 12.05, 12.10, 12.15, 12.20, 12.25, 12.30, 12.35, 12.40, 12.45, 12.50, 12.55, 12.60, 12.65, 12.70, 12.75, 12.80, 12.85, 12.90, 12.95, 13.00, 13.05, 13.10, 13.15, 13.20, 13.25, 13.30, 13.35, 13.40, 13.45, 13.50, 13.55, 13.60, 13.65, 13.70, 13.75, 13.80, 13.85, 13.90, 13.95, 14.00, 14.05, 14.10, 14.15, 14.20, 14.25, 14.30, 14.35, 14.40, 14.45, 14.50, 14.55, 14.60, 14.65, 14.70, 14.75, 14.80, 14.85, 14.90, 14.95, 15.00, 15.05, 15.10, 15.15, 15.20, 15.25, 15.30, 15.35, 15.40, 15.45, 15.50, 15.55, 15.60, 15.65, 15.70, 15.75, 15.80, 15.85, 15.90, 15.95, 16.00, 16.05, 16.10, 16.15, 16.20, 16.25, 16.30, 16.35, 16.40, 16.45, 16.50, 16.55, 16.60, 16.65, 16.70, 16.75, 16.80, 16.85, 16.90, 16.95, 17.00, 17.05, 17.10, 17.15, 17.20, 17.25, 17.30, 17.35, 17.40, 17.45, 17.50, 17.55, 17.60, 17.65, 17.70, 17.75, 17.80, 17.85, 17.90, 17.95, 18.00, 18.05, 18.10, 18.15, 18.20, 18.25, 18.30, 18.35, 18.40, 18.45, 18.50, 18.55, 18.60, 18.65, 18.70, 18.75, 18.80, 18.85, 18.90, 18.95, 19.00, 19.05, 19.10, 19.15, 19.20, 19.25, 19.30, 19.35, 19.40, 19.45, 19.50, 19.55, 19.60, 19.65, 19.70, 19.75, 19.80, 19.85, 19.90, 19.95, 20.00]

coefficient_trainee = [0.017,0.018,0.019,0.021,0.024,0.027,0.031,0.035,0.040,0.046,0.052,0.058,0.065,0.073,0.082,0.091,0.100,0.111,0.122,0.133,0.146,0.159,0.173,0.187,0.201,0.215,0.229,0.243,0.257,0.271,0.285,0.299,0.313,0.327,0.341,0.355,0.369,0.383,0.397,0.411,0.425,0.439,0.453,0.467,0.481,0.495,0.509,0.523,0.537,0.551,0.565,0.579,0.593,0.607,0.621,0.635,0.649,0.663,0.677,0.691,0.705,0.719,0.733,0.747,0.761,0.775,0.789,0.803,0.817,0.831,0.845,0.859,0.873,0.887,0.901,0.915,0.929,0.943,0.957,0.971,0.985,0.999,1.013,1.027,1.041,1.055,1.069,1.083,1.097,1.111,1.125,1.139,1.153,1.167,1.181,1.195,1.209,1.223,1.237,1.251,1.265,1.279,1.293,1.307,1.321,1.335,1.349,1.363,1.377,1.391,1.405,1.419,1.433,1.447,1.461,1.475,1.489,1.503,1.517,1.531,1.545,1.559,1.573,1.587,1.601,1.615,1.629,1.643,1.657,1.671,1.685,1.699,1.713,1.727,1.741,1.755,1.769,1.783,1.797,1.811,1.825,1.839,1.853,1.867,1.881,1.895,1.909,1.923,1.937,1.951,1.965,1.979,1.993,2.007,2.021,2.035,2.049,2.063,2.077,2.091,2.105,2.119,2.133,2.147,2.161,2.175,2.189,2.203,2.217,2.231,2.245,2.259,2.273,2.287,2.301,2.315,2.329,2.343,2.357,2.371,2.385,2.399,2.413,2.427,2.441,2.455,2.469,2.483,2.497,2.511,2.525,2.539,2.553,2.567,2.581,2.595,2.609,2.623,2.637,2.651,2.665,2.679,2.693,2.707,2.721,2.735,2.749,2.763,2.777,2.791,2.805,2.819,2.833,2.847,2.861,2.875,2.889,2.903,2.917,2.931,2.945,2.959,2.973,2.987,3.001,3.015,3.029,3.043,3.057,3.071,3.085,3.099,3.113,3.127,3.141,3.155,3.169,3.183,3.197,3.211,3.225,3.239,3.253,3.267,3.281,3.295,3.309,3.323,3.337,3.351,3.365,3.379,3.393,3.407,3.421,3.435,3.449,3.463,3.477,3.491,3.505,3.519,3.533,3.547,3.561,3.575,3.589,3.603,3.617,3.631,3.645,3.659,3.673,3.687,3.701,3.715,3.729,3.743,3.757,3.771,3.785,3.799,3.813,3.827,3.841,3.855,3.869,3.883,3.897,3.911,3.925,3.939,3.953,3.967,3.981,3.995,4.009,4.023,4.037,4.051,4.065,4.079,4.093,4.107,4.121,4.135,4.149,4.163,4.177,4.191,4.205,4.219,4.233,4.247,4.261,4.275,4.289,4.303,4.317,4.331,4.345,4.359,4.373,4.387,4.401,4.415,4.429,4.443,4.457,4.471,4.485,4.499,4.513,4.527,4.541,4.555,4.569,4.583,4.597,4.611,4.625,4.639,4.653,4.667,4.681,4.695,4.709,4.723,4.737,4.751,4.765,4.779,4.793,4.807,4.821,4.835,4.849,4.863,4.877,4.891,4.905,4.919,4.933,4.947,4.961,4.975,4.989,5.003,5.017,5.031,5.045,5.059,5.073,5.087,5.101,5.115,5.129,5.143,5.157,5.171,5.185,5.199,5.213,5.227,5.241,5.255,5.269,5.283,5.297,5.311,5.325,5.339,5.353,5.367,5.381,5.395,5.409,5.423,5.437,5.451,5.465,5.479,5.493,5.507,5.521,5.535,5.549,5.563,5.577,5.591,5.605,5.619,5.633,5.647,5.661,5.675,5.689,5.703,5.717,5.731,5.745,5.759,5.773,5.787,5.801,5.815,5.829,5.843,5.857,5.871,5.885,5.899,5.913,5.927,5.941,5.955,5.969,5.983,5.997,6.011,6.025,6.039,6.053,6.067,6.081,6.095,6.109,6.123,6.137,6.151,6.165,6.179,6.193,6.207,6.221,6.235,6.249,6.263,6.277,6.291,6.305,6.319,6.333,6.347,6.361,6.375,6.389,6.403,6.417,6.431,6.445,6.459,6.473,6.487,6.501,6.515,6.529,6.543,6.557,6.571,6.585,6.599,6.613,6.627,6.641,6.655,6.669,6.683,6.697,6.711,6.725,6.739,6.753,6.767,6.781,6.795,6.809,6.823,6.837,6.851,6.865,6.879,6.893,6.907,6.921,6.935,6.949,6.963,6.977,6.991,7.005,7.019,7.033,7.047,7.061,7.075,7.089,7.103,7.117,7.131,7.145,7.159,7.173,7.187,7.201,7.215,7.229,7.243,7.257,7.271,7.285,7.299,7.313,7.327,7.341,7.355,7.369,7.383,7.397,7.411,7.425,7.439,7.453,7.467,7.481,7.495,7.509,7.523,7.537,7.551,7.565,7.579,7.593,7.607,7.621,7.635,7.649,7.663,7.677,7.691,7.705,7.719,7.733,7.747,7.761,7.775,7.789,7.803,7.817,7.831,7.845,7.859,7.873,7.887,7.901,7.915,7.929,7.943,7.957,7.971,7.985,7.999,8.013,8.027,8.041,8.055,8.069,8.083,8.097,8.111,8.125,8.139,8.153,8.167,8.181,8.195,8.209,8.223,8.237,8.251,8.265,8.279,8.293,8.307,8.321,8.335,8.349,8.363,8.377,8.391,8.405,8.419,8.433,8.447,8.461,8.475,8.489,8.503,8.517,8.531,8.545,8.559,8.573,8.587,8.601,8.615,8.629,8.643,8.657,8.671,8.685,8.699,8.713,8.727,8.741,8.755,8.769,8.783,8.797,8.811,8.825,8.839,8.853,8.867,8.881,8.895,8.909,8.923,8.937,8.951,8.965,8.979,8.993,9.007,9.021,9.035,9.049,9.063,9.077,9.091,9.105,9.119,9.133,9.147,9.161,9.175,9.189,9.203,9.217,9.231,9.245,9.259,9.273,9.287,9.301,9.315,9.329,9.343,9.357,9.371,9.385,9.399,9.413,9.427,9.441,9.455,9.469,9.483,9.497,9.511,9.525,9.539,9.553,9.567,9.581,9.595,9.609,9.623,9.637,9.651,9.665,9.679,9.693,9.707,9.721,9.735,9.749,9.763,9.777,9.791,9.805,9.819,9.833,9.847,9.861,9.875,9.889,9.903,9.917,9.931,9.945,9.959,9.973,9.987,10.001,10.015,10.029,10.043,10.057,10.071,10.085,10.099,10.113,10.127,10.141,10.155,10.169,10.183,10.197,10.211,10.225,10.239,10.253,10.267,10.281,10.295,10.309,10.323,10.337,10.351,10.365,10.379,10.393,10.407,10.421,10.435,10.449,10.463,10.477,10.491,10.505,10.519,10.533,10.547,10.561,10.575,10.589,10.603,10.617,10.631,10.645,10.659,10.673,10.687,10.701,10.715,10.729,10.743,10.757,10.771,10.785,10.799,10.813,10.827,10.841,10.855,10.869,10.883,10.897,10.911,10.925,10.939,10.953,10.967,10.981,10.995,11.009,11.023,11.037,11.051,11.065,11.079,11.093,11.107,11.121,11.135,11.149,11.163,11.177,11.191,11.205,11.219,11.233,11.247,11.261,11.275,11.289,11.303,11.317,11.331,11.345,11.359,11.373,11.387,11.401,11.415,11.429,11.443,11.457,11.471,11.485,11.499,11.513,11.527,11.541,11.555,11.569,11.583,11.597,11.611,11.625,11.639,11.653,11.667,11.681,11.695,11.709,11.723,11.737,11.751,11.765,11.779,11.793,11.807,11.821,11.835,11.849,11.863,11.877,11.891,11.905,11.919,11.933,11.947,11.961,11.975,11.989,12.003,12.017,12.031,12.045,12.059,12.073,12.087,12.101,12.115,12.129,12.143,12.157,12.171,12.185,12.199,12.213,12.227,12.241,12.255,12.269,12.283,12.297,12.311,12.325,12.339,12.353,12.367,12.381,12.395,12.409,12.423,12.437,12.451,12.465,12.479,12.493,12.507,12.521,12.535,12.549,12.563,12.577,12.591,12.605,12.619,12.633,12.647,12.661,12.675,12.689,12.703,12.717,12.731,12.745,12.759,12.773,12.787,12.801,12.815,12.829,12.843,12.857,12.871,12.885,12.899,12.913,12.927,12.941,12.955,12.969,12.983,12.997,13.011,13.025,13.039,13.053,13.067,13.081,13.095,13.109,13.123,13.137,13.151,13.165,13.179,13.193,13.207,13.221,13.235,13.249,13.263,13.277,13.291,13.305,13.319,13.333,13.347,13.361,13.375,13.389,13.403,13.417,13.431,13.445,13.459,13.473,13.487,13.501,13.515,13.529,13.543,13.557,13.571,13.585,13.599,13.613,13.627,13.641,13.655,13.669,13.683,13.697,13.711,13.725,13.739,13.753,13.767,13.781,13.795,13.809,13.823,13.837,13.851,13.865,13.879,13.893,13.907,13.921,13.935,13.949,13.963,13.977,13.991,14.005,14.019,14.033,14.047,14.061,14.075,14.089,14.103,14.117,14.131,14.145,14.159,14.173,14.187,14.201,14.215,14.229,14.243,14.257,14.271,14.285,14.299,14.313,14.327,14.341,14.355,14.369,14.383,14.397,14.411,14.425,14.439,14.453,14.467,14.481,14.495,14.509,14.523,14.537,14.551,14.565,14.579,14.593,14.607,14.621,14.635,14.649,14.663,14.677,14.691,14.705,14.719,14.733,14.747,14.761,14.775,14.789,14.803,14.817,14.831,14.845,14.859,14.873,14.887,14.901,14.915,14.929,14.943,14.957,14.971,14.985,14.999,15.013,15.027,15.041,15.055,15.069,15.083,15.097,15.111,15.125,15.139,15.153,15.167,15.181,15.195,15.209,15.223,15.237,15.251,15.265,15.279,15.293,15.307,15.321,15.335,15.349,15.363,15.377,15.391,15.405,15.419,15.433,15.447,15.461,15.475,15.489,15.503,15.517,15.531,15.545,15.559,15.573,15.587,15.601,15.615,15.629,15.643,15.657,15.671,15.685,15.699,15.713,15.727,15.741,15.755,15.769,15.783,15.797,15.811,15.825,15.839,15.853,15.867,15.881,15.895,15.909,15.923,15.937,15.951,15.965,15.979,15.993,16.007,16.021,16.035,16.049,16.063,16.077,16.091,16.105,16.119,16.133,16.147,16.161,16.175,16.189,16.203,16.217,16.231,16.245,16.259,16.273,16.287,16.301,16.315,16.329,16.343,16.357,16.371,16.385,16.399,16.413,16.427,16.441,16.455,16.469,16.483,16.497,16.511,16.525,16.539,16.553,16.567,16.581,16.595,16.609,16.623,16.637,16.651,16.665,16.679,16.693,16.707,16.721,16.735,16.749,16.763,16.777,16.791,16.805,16.819,16.833,16.847,16.861,16.875,16.889,16.903,16.917,16.931,16.945,16.959,16.973,16.987,17.001,17.015,17.029,17.043,17.057,17.071,17.085,17.099,17.113,17.127,17.141,17.155,17.169,17.183,17.197,17.211,17.225,17.239,17.253,17.267,17.281,17.295,17.309,17.323,17.337,17.351,17.365,17.379,17.393,17.407,17.421,17.435,17.449,17.463,17.477,17.491,17.505,17.519,17.533,17.547,17.561,17.575,17.589,17.603,17.617,17.631,17.645,17.659,17.673,17.687,17.701,17.715,17.729,17.743,17.757,17.771,17.785,17.799,17.813,17.827,17.841,17.855,17.869,17.883,17.897,17.911,17.925,17.939,17.953,17.967,17.981,17.995,18.009,18.023,18.037,18.051,18.065,18.079,18.093,18.107,18.121,18.135,18.149,18.163,18.177,18.191,18.205,18.219,18.233,18.247,18.261,18.275,18.289,18.303,18.317,18.331,18.345,18.359,18.373,18.387,18.401,18.415,18.429,18.443,18.457,18.471,18.485,18.499,18.513,18.527,18.541,18.555,18.569,18.583,18.597,18.611,18.625,18.639,18.653,18.667,18.681,18.695,18.709,18.723,18.737,18.751,18.765,18.779,18.793,18.807,18.821,18.835,18.849,18.863,18.877,18.891,18.905,18.919,18.933,18.947,18.961,18.975,18.989,19.003,19.017,19.031,19.045,19.059,19.073,19.087,19.101,19.115,19.129,19.143,19.157,19.171,19.185,19.199,19.213,19.227,19.241,19.255,19.269,19.283,19.297,19.311,19.325,19.339,19.353,19.367,19.381,19.395,19.409,19.423,19.437,19.451,19.465,19.479,19.493,19.507,19.521,19.535,19.549,19.563,19.577,19.591,19.605,19.619,19.633,19.647,19.6
```

```

listes_des_portances=[]
listes_des_trainees=[]

#coordonée de l aile
extrados_x = [-0.514,-0.5075,-0.48825,-0.45675,-0.4135,-0.35925,-0.29525,-0.22275,-0.14325,-0.05875,0.02875,0.11675,0.20275,0.28325,0.3555,0.416,0.46175,0.4902]
extrados_y = [0, 0.0185, 0.03575, 0.051, 0.06325, 0.07175, 0.07625, 0.0765, 0.07275, 0.0655, 0.0555, 0.044, 0.032, 0.021, 0.012, 0.0055, 0.00175, 0.00025, 0]

intrados_x =[-0.514, -0.5075, -0.48825, -0.45675, -0.4135, -0.35925, -0.29525, -0.22275, -0.14325, -0.05875, 0.02875, 0.11675, 0.20275, 0.28325, 0.3555, 0.416,
intrados_y =[0, -0.0185, -0.03575, -0.051, -0.06325, -0.07175, -0.07625, -0.0765, -0.07275, -0.0655, -0.0555, -0.044, -0.032, -0.021, -0.012, -0.0055, -0.00175, 0]

##Pression autour de l aile

pression_extrados = [102.511, 101.782, 101.001, 100.664, 100.558, 100.562, 100.623, 100.716, 100.826, 100.942, 101.058, 101.168, 101.269, 101.358, 101.432, 101
pression_intrados = [102.511, 101.782, 101.001, 100.664, 100.558, 100.562, 100.623, 100.716, 100.826, 100.942, 101.058, 101.168, 101.269, 101.358, 101.432, 10
#PRESSION MOIN 1 atm

##Fonctions ( calculs de l aire et du perimetre de l aile )

def distance( x1, y1, x2, y2):
    return np.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

def calcul_perimetre(x_ext, y_ext):
    perimetre = 0

    for i in range(len(x_ext) - 1):
        perimetre += distance(x_ext[i], y_ext[i], x_ext[i + 1], y_ext[i + 1])
    return perimetre

def calcul_aire_interne(x_int, y_int, x_ext, y_ext):
    aire_interne = 0
    for i in range(len(x_int) - 1):
        aire_interne += 0.5 * (x_int[i + 1] - x_int[i]) * (y_ext[i] + y_ext[i + 1] - y_int[i] - y_int[i + 1])
    return abs(aire_interne)

def calcul_aire_ailaire(x, y):

    aire_ailaire = 0
    for i in range(len(x) - 1):
        aire_ailaire += 0.5 * (x[i + 1] - x[i]) * (y[i] + y[i + 1]) # Somme des trapèzes
    return abs(aire_ailaire)

## Formules

perimetre = calcul_perimetre(extrados_x, extrados_y)

Aire_interne = calcul_aire_interne(intrados_x, intrados_y, extrados_x, extrados_y)

Aire_ailaire = perimetre * longueur_aile # c est pour 1 SEUL aile (oui car plan * longueur (axe y) = 3Dim)

print("l'aire de l aile est de :",round(Aire_ailaire,3) ,"m^2","pour un périmetre de :",round(perimetre,3),"m" )

## Calcul des portances du drones ( creer)

for cp, ct in zip(coefficient_portance, coefficient_trainee):
    portance = cp * 0.5 * ro * (vitesse**2) * Aire_ailaire
    trainee = ct * 0.5 * ro * (vitesse**2) * Aire_ailaire
    portances.append(portance)
    trainees.append(trainee)

```

```

# Trouver la valeur maximale de la portance et l'angle correspondant avec la portance a cet angles
max_portance = max(portances)

max_index = portances.index(max_portance)
angle_max_portance = angles[max_index]
trainee_max_portance = round(trainees[max_index],2)
force_max_utile=round(max_portance-trainee_max_portance,1)

cp_angle15_aile= coefficient_portance[max_index]

# Afficher uniquement le coefficient de portance correspondant
coefficient_portance[max_index]

print("portance maximale :", round(max_portance,3),

      "pour un angle de :", angle_max_portance,
      " et une trainée :", trainee_max_portance,
      "soit une force max :", force_max_utile)
## histogrammes
bar_width = 0.5
indices = np.arange(len(angles))
plt.figure(figsize=(12, 6))

# Histogramme pour la portance
plt.bar(indices, portances, bar_width, label='Portance', color='blue')

# Histogramme pour la trainée, légèrement décalé pour qu'ils ne se superposent pas
plt.bar(indices + bar_width, trainees, bar_width, label='Trainée', color='red')
plt.axvline(max_index, color='green', linestyle='--', label=f"Portance Max à {angle_max_portance}°")

```

```

'''
#Erreur Corrigé au code 3

##COMPARAISON des modeles ( traditionnel a ceux crée)
#( on regarde la puissance nécessaire pour etre l equilibre par une aile)

#angle désiré:      1rad * 180/n = 57,296°
                   # 0.0174533 rad          = 1°

angle_désiré_en_radian = 0.0174533 * (90- angle_max_portance)      # on veut 15 degré ici en radian

portance_bras_ailé = max_portance

#Drone traditionnel a 4 moteur :

vitesse_air_equilibre_trad = ( (m*g*2)/(ro* aire_ailaire_petite_aile *coefficient_portance_petite_aile) )**(1/2)
Portance_petite_aile_equilibre = (m*g)                                #portance pour une unique aile (sur un total de 4)
Puissance_traditionnel = Portance_petite_aile_equilibre * vitesse_air_equilibre_trad
print("vitesse trad ",vitesse_air_equilibre_trad)

#oui c est correct pour le 1 er modele .

#Drone SANS bras ailés a 15 ° :

vitesse_air_equilibre_angle = ( (m*g*2)/(ro* aire_ailaire_petite_aile*p *coefficient_portance_petite_aile*np.cos(angle_désiré_en_radian)) )**(1/2)

print("vitesse sans ",vitesse_air_equilibre_angle)

Puissance_Sans_bras_ailé = ((m*g) ) *np.cos(angle_désiré_en_radian) *n * vitesse_air_equilibre_angle      # a verif

#Drone AVEC bras ailés a 15 ° :

vitesse_air_equilibre_angle_AVEC_ailes = ( (m*g*2)/(ro* (aire_ailaire_petite_aile*p*(coefficient_portance_petite_aile) + (Aire_ailaire* cp_angle15_aile) ) *np.

#on suppose que vitesse de la air est uniforme sur tout le bras (aile + rotor)

```

```

print("vitesse avec ",vitesse_air_equilibre_angle_AVEC_ailes)

#ici on a la puissance produite
Puissance_Avec_bras_aile = portance_bras_aile*n* vitesse_air_equilibre_angle_AVEC_ailes # VRAI TKT

Puissance_Avec_bras_aile_rotor = p*n* rp* aire_ailaire_petite_aile *(vitesse_air_equilibre_angle_AVEC_ailes**2)*coefficient_portance_petite_aile

PUISSANCE_Théorique = portance_bras_aile *vitesse *n

def vitesse_radiale(v_m_s, rayon):
    # Conversion de la vitesse linéaire à une vitesse angulaire en radians par seconde
    omega = v_m_s / rayon #  $\omega = v / r$ 
    return (omega)

print(
    "trad:", round(vitesse_radiale(vitesse_air_equilibre_trad, 0.03), 2),
    "sans ail:", round(vitesse_radiale(vitesse_air_equilibre_angle, 0.03), 2),
    "avc aile :", round(vitesse_radiale(vitesse_air_equilibre_angle_AVEC_ailes, 0.03), 2) ,
    "en rad/seconde !:"
)

##Print généraux

print("=====ENERGIE NECESSAIRE AU VOL Des DRONES PAR AILES a",angle_max_portance ,"" , "=====")

print("puissance minimal électrique au vol d'un drone traditionnel:      * ,round(Puissance_traditionnel,2),\"Watts\")

print("puissance minimal électrique au vol d'un drone SANS bras aile : ",round(Puissance_Sans_bras_aile,2),\"Watts\")

print("puissance minimal électrique au vol d'un drone AVEC bras ailes:      * ,round(Puissance_Avec_bras_aile_rotor,2)
,\"Watts\")

print("puissance total produite électrique au vol d'un drone AVEC bras ailes:      * ,round(Puissance_Avec_bras_aile,2)
,\"Watts\")

print("=====")

print("la portance maximal est de * , round(max_portance,3),\"Newtons\", \"pour un angle max\",angle_max_portance , \"et une valeur de trainée de\" ,round(trainee_max_p
\"soit un force max utile de\" ,force_max_utile,\"N\" )

print("
    ")
print("Puissance produite :\" ,PUISSANCE_Théorique,\"W\" )   ***

```

```
#histogramme : donnée
```

```
#Annotation sur le graphique des portances et de la trainée
point_a_annoter = list(range(0, len(angles), 1))
angles_np = np.array(angles) # Conversion en tableau NumPy
portances_np = np.array(portances)
trainees_np = np.array(trainees)
# Points sur les barres pour la portance et la trainée
plt.scatter(indices, portances_np, zorder=5)
plt.scatter(indices + bar_width, trainees_np, zorder=5)

# Ajouter des annotations sur chaque point AVEC un décalage en escalier
for i in range(len(angles_np)):
    if i % 3 == 0:
        offset = -10
    elif i % 3 == 1:
        offset = 5
    else:
        offset = +15 # Décalage alternatif pour éviter le chevauchement
    plt.annotate(f'({portances_np[i]:.1f})',
                (indices[i], portances_np[i]),
                textcoords="offset points", xytext=(0, offset), ha='center', fontsize=8, color='orange', zorder=10)
    plt.annotate(f'({trainees_np[i]:.1f})',
                (indices[i] + bar_width, trainees_np[i]),
                textcoords="offset points", xytext=(0, offset), ha='center', fontsize=8, color='cyan', zorder=10)

# Labels et titre de l histogramme
plt.xlabel('Angle d\'attaque (degrés)')
plt.ylabel('Force (N)')
plt.title('Histogramme des forces de Portance et de Trainée en fonction de l\'angle d\'attaque')
plt.xticks(indices + bar_width / 2, angles, rotation=45)
plt.legend()
plt.grid(True)

# Affichage

##courbes 2
# Affichage des résultats

#print("Portances:", portances)
#print("Trainées:", trainees)

plt.figure(figsize=(9, 6))

plt.plot(angles, portances, label='Portance', color='blue')
plt.plot(angles, trainees, label='Trainée', color='red')
plt.xlabel('Angle d\'attaque (degrés)')

plt.ylabel('Force (N)')
plt.title('Portance et Trainée en fonction de l\'angle d\'attaque')
plt.legend()
plt.grid(True)

##
#courbe
plt.figure(figsize=(5, 5))
plt.plot(extrados_x, extrados_y, label='Extrados', color='black')
plt.plot(intrados_x, intrados_y, label='Intrados', color='blue')
```

```

#Séparer les vecteurs en fonction des signes des composantes (Pression vectorielle )
# #for i in range(len(pression_extrados)):
#     plt.quiver(extrados_x[i], extrados_y[i], pression_extrados[i], extrados_x[i], color="purple", angles='xy', scale_units='xy', scale=(1500))
#
# #for i in range(len(pression_intrados)):
#     plt.quiver(intrados_x[i], intrados_y[i], pression_intrados[i], intrados_x[i], color="orange", angles='xy', scale_units='xy', scale=(1500))

plt.title("Cartographie des pression autour de l aile ")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.axis('equal')
plt.legend()
plt.show

import matplotlib.pyplot as plt
import numpy as np

def rapport (angles_np, portances_np, trainees_np):
    rapport_np = portances_np / trainees_np

    # Recherche du maximum et de l'angle correspondant
    max_rapport = np.max(rapport_np)
    indice_max = np.argmax(rapport_np)
    angle_optimal = angles_np[indice_max]

    # Affichage de la courbe
    plt.figure(figsize=(8, 6))
    plt.plot(angles_np, rapport_np, marker='s', color='green', label='Portance / Trainée')

    # Marquage du point optimal
    plt.scatter(angle_optimal, max_rapport, color='red', zorder=5, label=f"Max à {angle_optimal}°")
    plt.annotate(f"Max: {max_rapport:.2f} \n à {angle_optimal}°",
                (angle_optimal, max_rapport),
                textcoords="offset points", xytext=(10, -15), ha='left',
                fontsize=9, color='red')

    # Légendes et mise en forme
    plt.xlabel("Angle d'incidence (°)")
    plt.ylabel("Portance / Trainée")
    plt.title("Rendement aérodynamique en fonction de l'incidence")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()

    plt.plot(angles_np, portances_np, label="Portance", marker='o')
    plt.plot(angles_np, trainees_np, label="Trainée", marker='x')
    plt.legend()
    plt.xlabel("Angle (°)")
    plt.ylabel("Coefficient")
    plt.title("Portance et Trainée en fonction de l'angle")
    plt.grid(True)
    plt.show()

rapport (angles_np, portances_np, trainees_np)

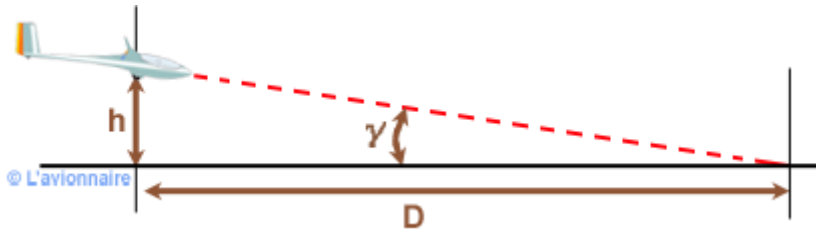
```

```

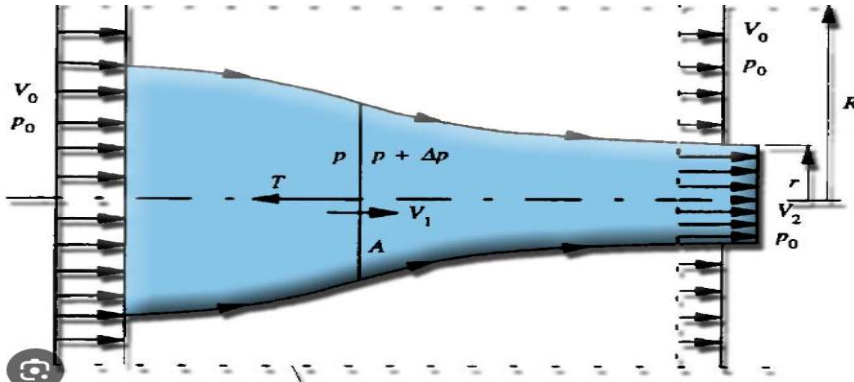
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Constantes physiques et géométriques
5 masse = 1.5 # en kg
6 g = 9.81 # m/s²
7 rho = 1.225 # kg/m³ (air à 15°C)
8 S = 0.3 # Surface alaire en m²
9
10 # Plages de variations
11 vites = np.linspace(1, 30, 100) # vitesses de 1 à 30 m/s
12 angles_deg = np.linspace(0, 90, 100) # angles de 0° à 90°
13 angles_rad = np.radians(angles_deg)
14
15 # 1. CL en fonction de la vitesse pour différents angles
16 plt.figure(figsize=(10, 5))
17 for angle_deg in [0, 15, 30, 45, 60, 75, 90]:
18     theta = np.radians(angle_deg)
19     CL = (masse * g * np.cos(theta)) / (2 * rho * S * vites**2)
20     plt.plot(vites, CL, label=f"{angle_deg}°")
21
22 plt.title("Coefficient de portance (CL) en fonction de la vitesse")
23 plt.xlabel("Vitesse (m/s)")
24 plt.ylabel("C_L")
25 plt.grid(True)
26 plt.legend(title="Angle (°)")
27 plt.tight_layout()
28 plt.show()
29
30 # 2. CL en fonction de l'angle pour différentes vitesses
31 plt.figure(figsize=(10, 5))
32 for V in [5, 10, 15, 20, 25]:
33     CL = (masse * g * np.cos(angles_rad)) / (2 * rho * S * V**2)
34     plt.plot(angles_deg, CL, label=f"{V} m/s")
35
36 plt.title("Coefficient de portance (CL) en fonction de l'angle")
37 plt.xlabel("Angle (°)")
38 plt.ylabel("C_L")
39 plt.grid(True)
40 plt.legend(title="Vitesse")
41 plt.tight_layout()
42 plt.show()
43

```

Sources :



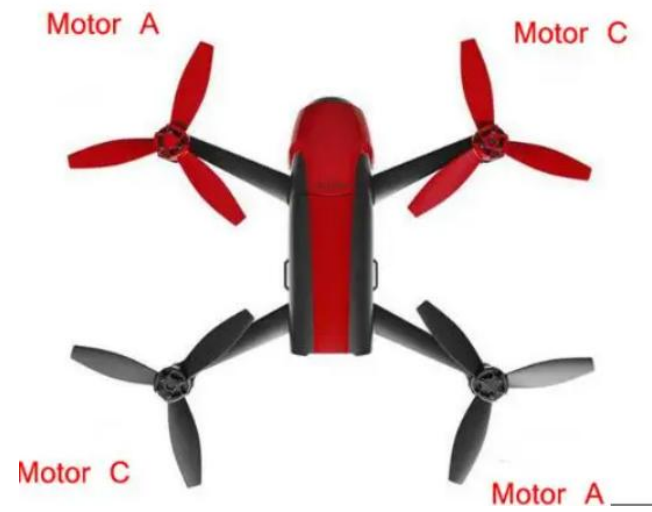
<https://www.google.com/imgres?q=l%27avionnaire%20finesse&imgurl=https%3A%2F%2Fwww.lavionnaire.fr%2FSiteImgPlaneur%2FFinesseDessin.png&imgrefurl=https%3A%2F%2Fwww.lavionnaire.fr%2FPlaneurPolaires.php&docid=f7fEARYk77NDIM&tbid=F7X-TgNyhjRtvM&vet=12ahUKEwjtu-LnjOaNaxWsdaQEHeMtDloQM3oECFQQAA..i&w=410&h=116&hcb=2&ved=2ahUKEwjtu-LnjOaNaxWsdaQEHeMtDloQM3oECFQQAA>



<https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.heliciel.com%2Fhelice%2Fcalcul-helice-aile%2Ftheorie%2520de%2520froude%2520relative%2520aux%2520helices%2520de%2520traction%2520ou%2520propulsion.htm&psig=AOvVaw3Vwa6aPI6kEvBTvjQElxP&ust=1749619235391000&source=images&cd=vfe&opi=89978449&ved=0CBUQjRxqFwoTCKj46dmN5o0DFQAAAAAdAAAAABAE>



www.youtube.com/watch?v=7JH1_ZKV7t4



<https://www.google.com/url?sa=i&url=https%3A%2F%2Ffr.aliexpress.com%2Fitem%2F32650982670.html&psig=AOvVaw3PXZ3QvQEgDzzRUjVFI-WJ&ust=1749621622082000&source=images&cd=vfe&opi=89978449&ved=0CBUQjRxqFwoTCPjx6s6W5o0DFQAAAAAdAAAAABAE>



https://www.google.com/url?sa=i&url=https%3A%2F%2Ftoulouse.la Tribune.fr%2Finnovation%2F2021-03-25%2Favec-tindair-toulouse-et-bordeaux-vont-tester-l-insertion-de-drones-en-milieu-urbain-880852.html&psig=AOvVaw2T-gtCPmVm_nS8FkZt5n4&ust=1749628866410000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCODu18qx5o0DFQAAAAAdAAAAABAE